Architecture des microprocesseur

Module SLEEL602

Cours de deuxième semestre

L3 Electronique

Pré-requis : module SPUE42

LABORATOIRE D'ELECTRONIQUE
ANTENNES ET TELECOMMUNICATIONS

UNIVERSITÉ
CÔTE D'AZUR

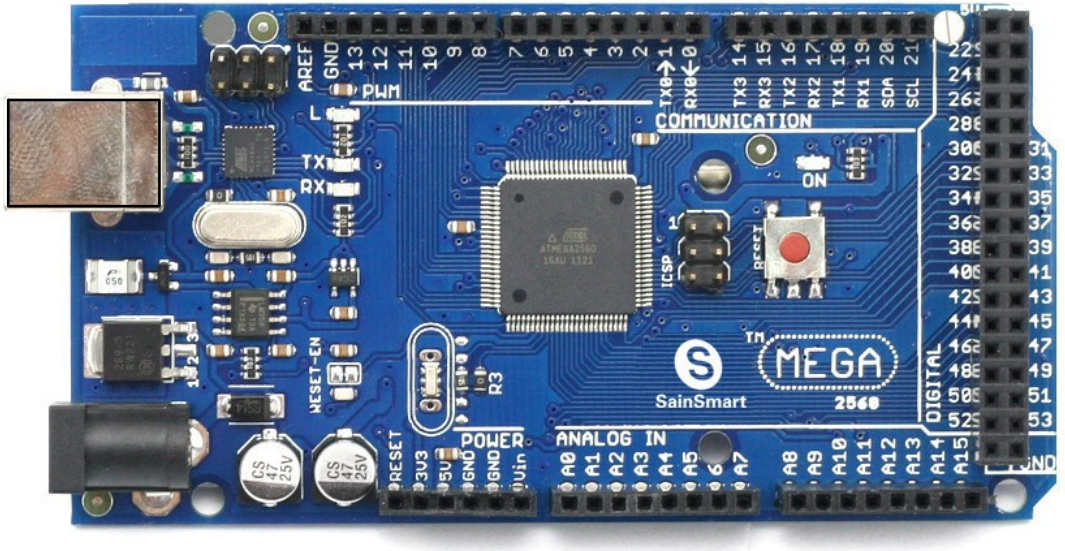
François VERDIER
Laboratoire LEAT
Université Côte d@Azur

Plan du cours

1. Introduction
2. Les machines de calcul
3. Les branchements conditionnels
4. **Architecture du micro-contrôleur ATmega2560**
    1. **Les Microcontrôleurs AVR**
    2. **Les Microcontrôleurs de type ATmega2560**
    3. **Les ressources internes du CPU AVR**
    4. **Les 13 modes d'adressage**
    5. **Le jeu d'instruction de l'AVR ATmega2560**
    6. **Le brochage de la carte Arduino ATmega2560**
5. Les interruptions
6. Les périphériques du micro-contrôleur

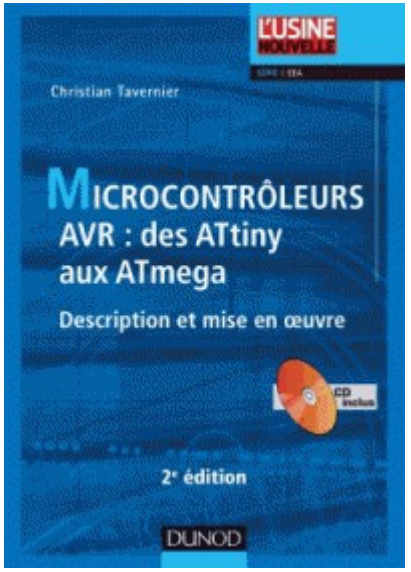
Chapitre 4 :

Architecture du micro-contrôleur ATmega2560



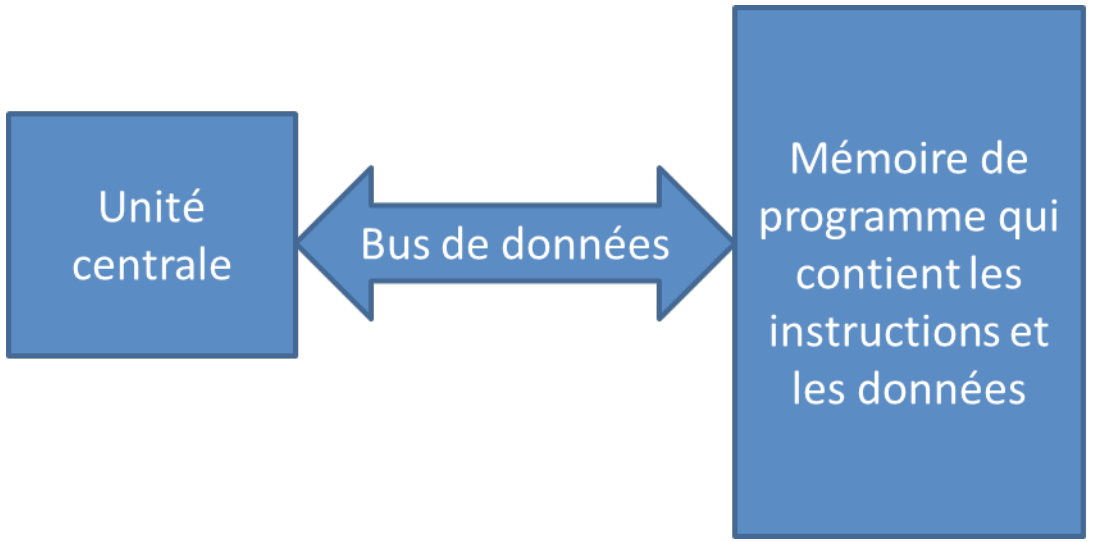
Bibliographie

- Micro-Contrôleurs AVR : des ATtiny aux ATmega. Description et mise en œuvre. 2ème édition. Christian Tavernier. L'usine nouvelle, DUNOD, 2013



1. Les Microcontrôleurs AVR

- Les Microprocesseurs sont généralement des machines dites « de Von Neumann »



Les avantages :

- Architecture très classique
- Architecture très simple

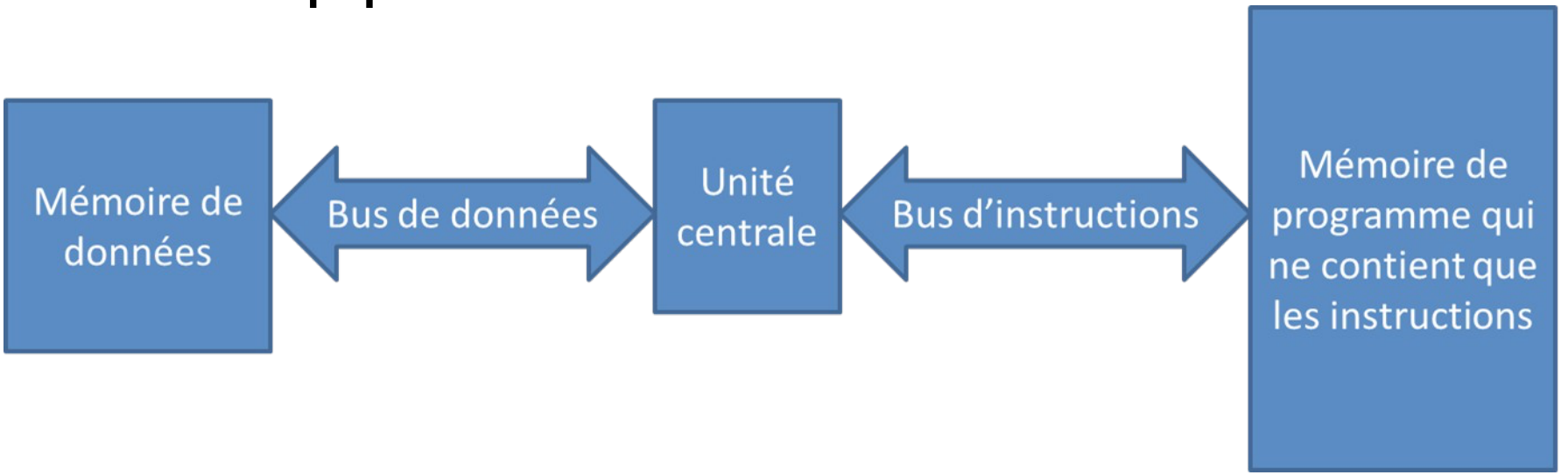
Les inconvénients :

- Le processeur ne peut aller chercher qu'une seule instruction ou une seul donnée à la fois !

Architecture de type Von Neumann où la mémoire contient à la fois les données et les instructions

1. Les Microcontrôleurs AVR

- Tous les micro-contrôleurs Arduino sont basés sur des architectures particulières de type Harvard, RISC et pipeline



Architecture de type HARVARD où les données et les instructions sont placées dans des mémoires séparées.

Avantage : une bien meilleure performance dans les accès mémoire

1. Les Microcontrôleurs AVR

- Le **pipeline** est une technique dans laquelle plusieurs instructions se chevauchent lors de l'exécution. Le pipeline est divisé en étapes et ces étapes sont reliées les unes aux autres pour former un tuyau ressemblant à une structure. Les instructions entrent d'un bout et sortent d'un autre bout.
- Le traitement en **pipeline** augmente le débit d'instruction global.
- Dans un système en **pipeline**, chaque segment est constitué d'un registre d'entrée suivi d'un circuit combinatoire. Le registre est utilisé pour conserver des données et le circuit combinatoire effectue des opérations dessus. La sortie du circuit combinatoire est appliquée au registre d'entrée du segment suivant.

Illustration d'un fonctionnement classique

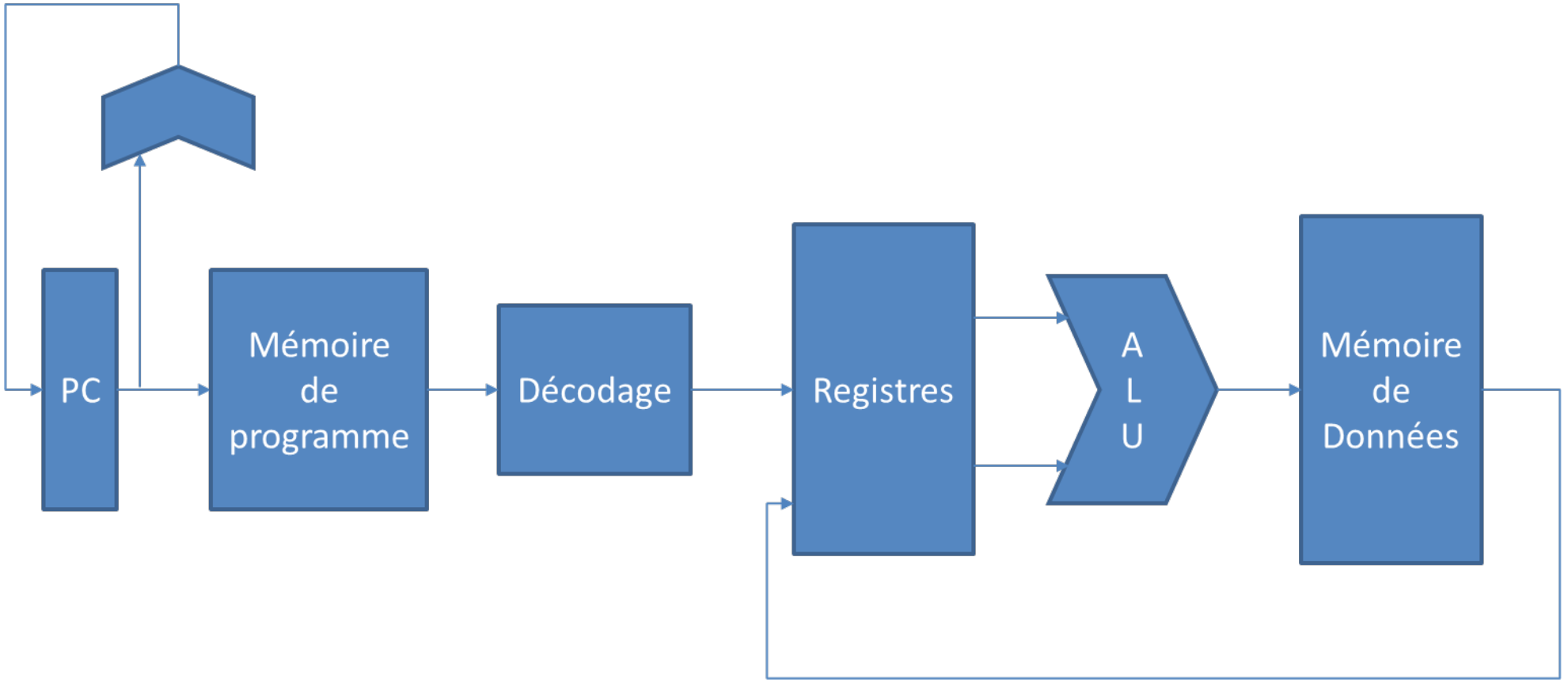


Illustration d'un fonctionnement classique

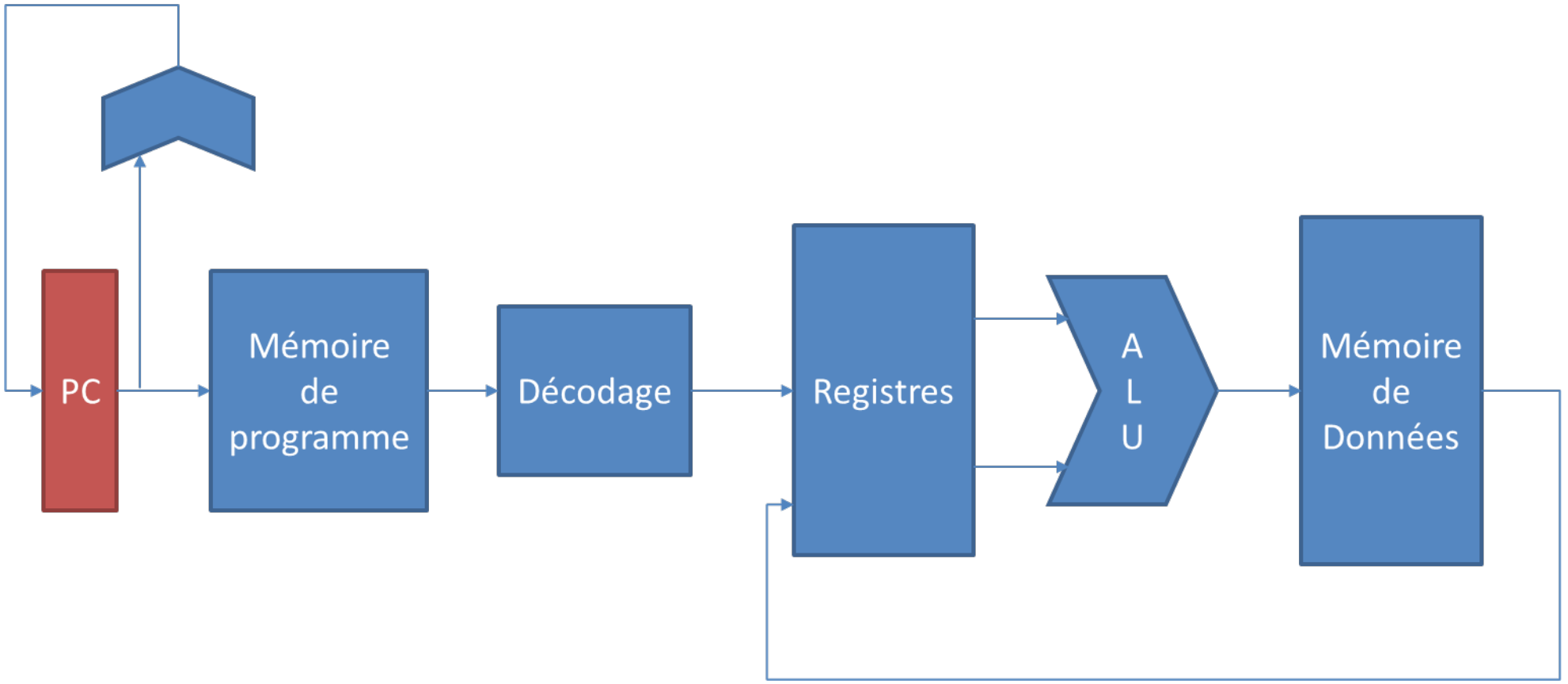


Illustration d'un fonctionnement classique

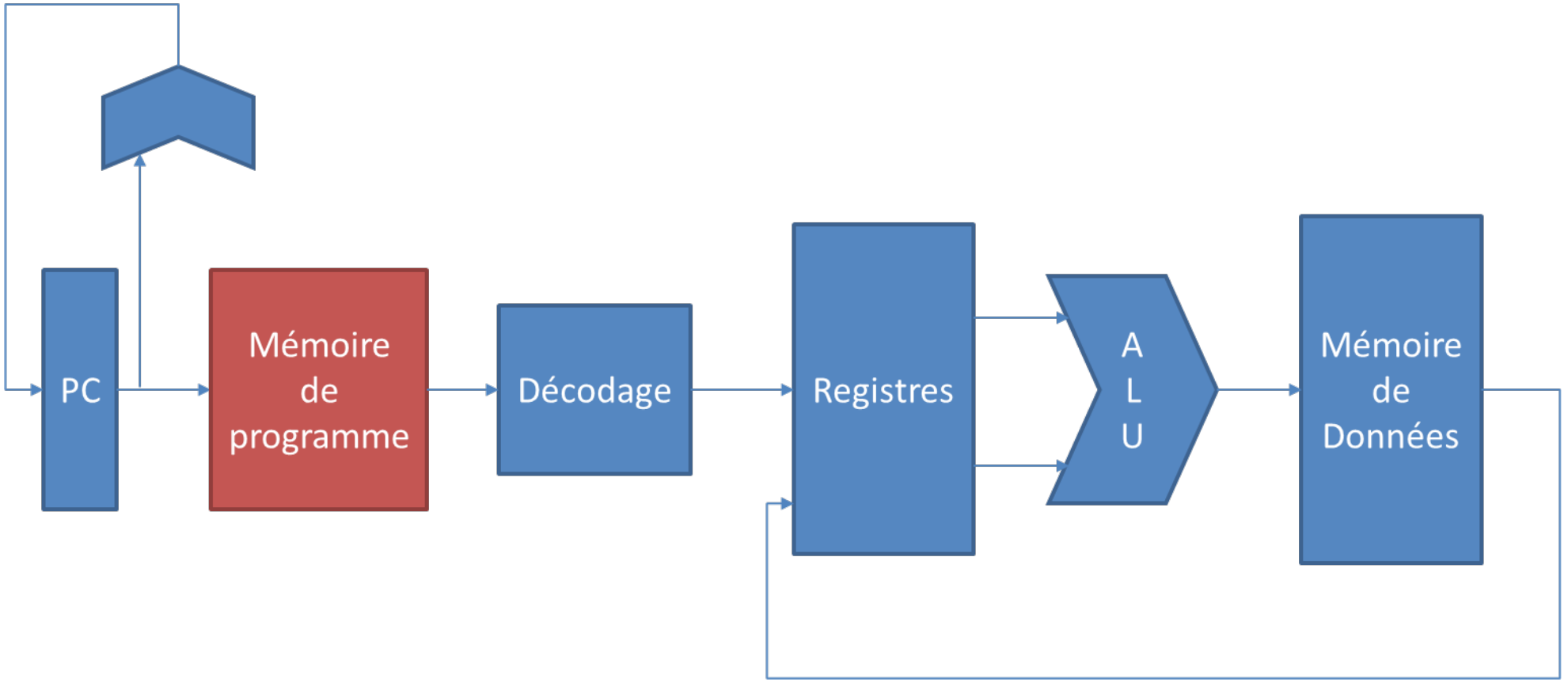


Illustration d'un fonctionnement classique

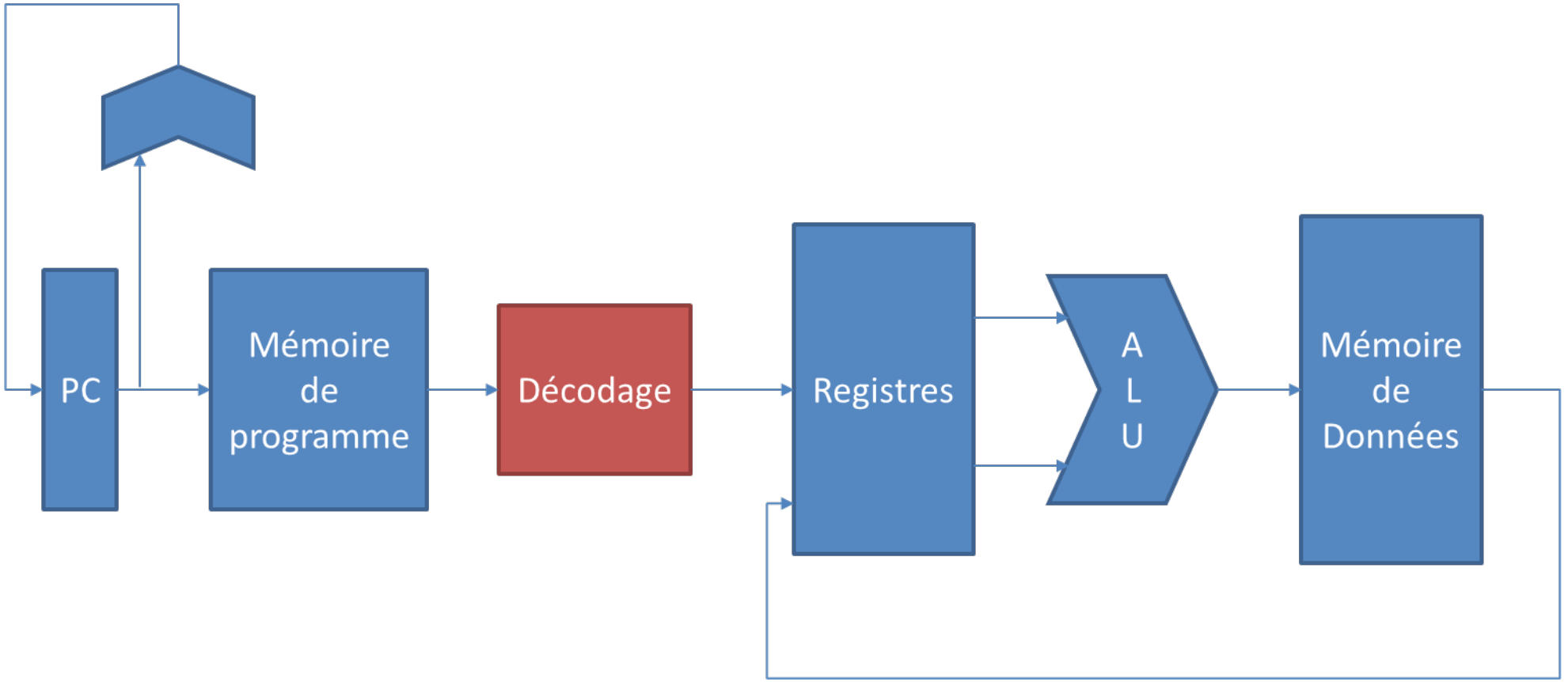


Illustration d'un fonctionnement classique

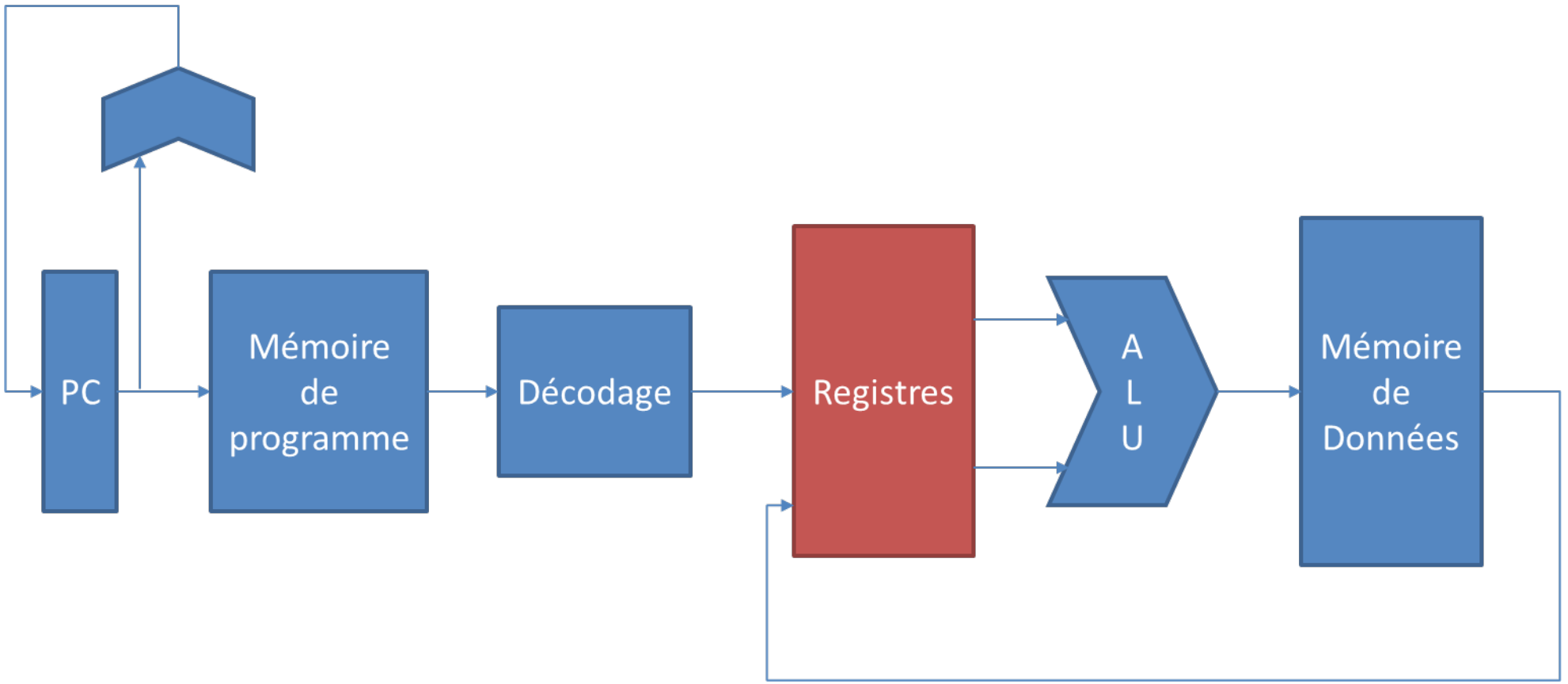


Illustration d'un fonctionnement classique

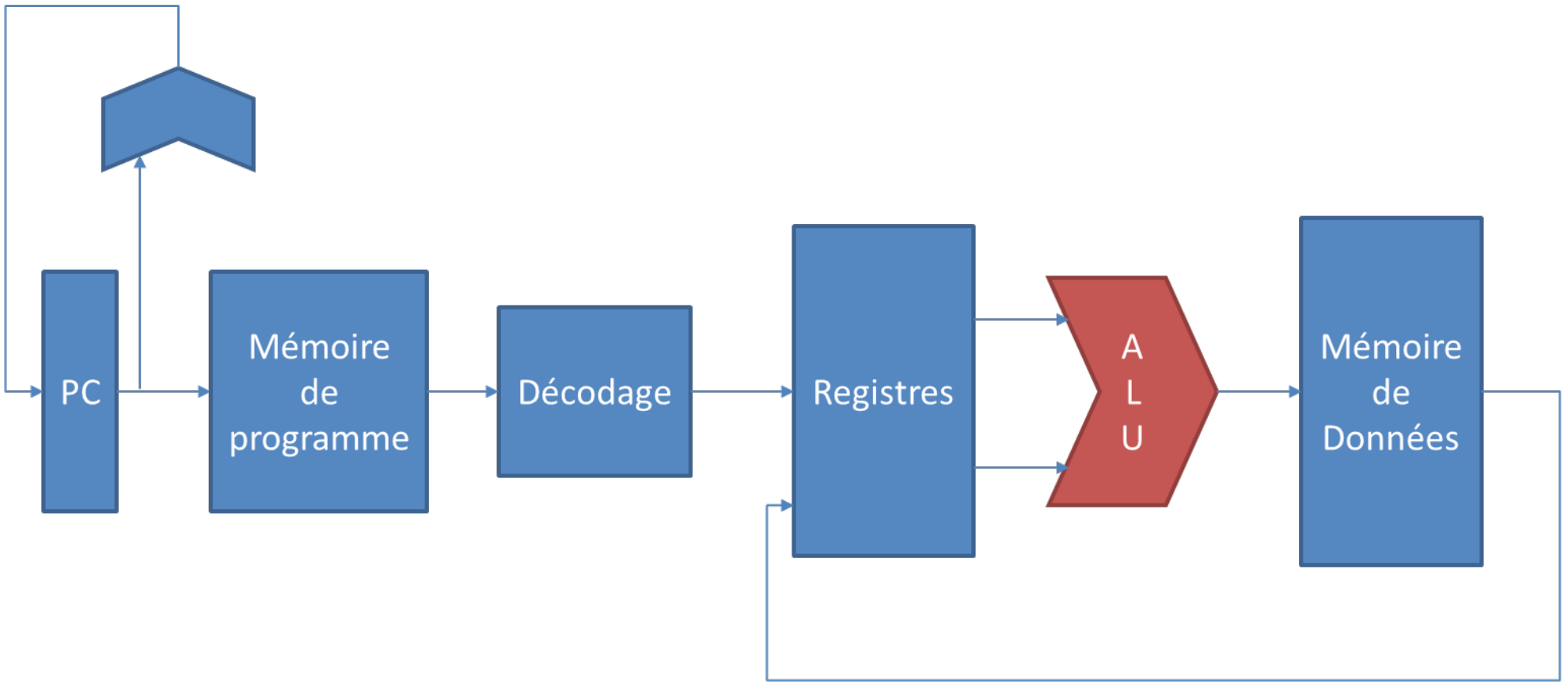


Illustration d'un fonctionnement classique

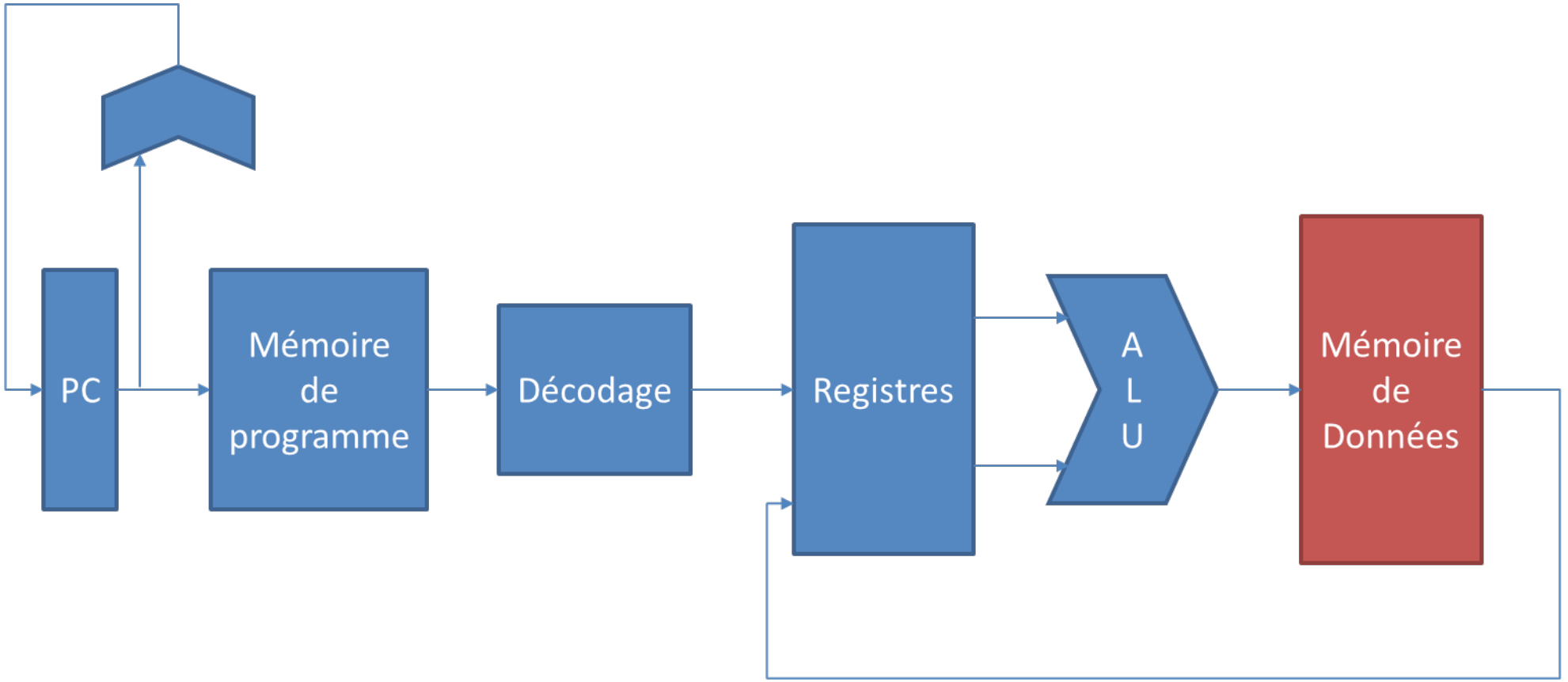


Illustration d'un fonctionnement classique

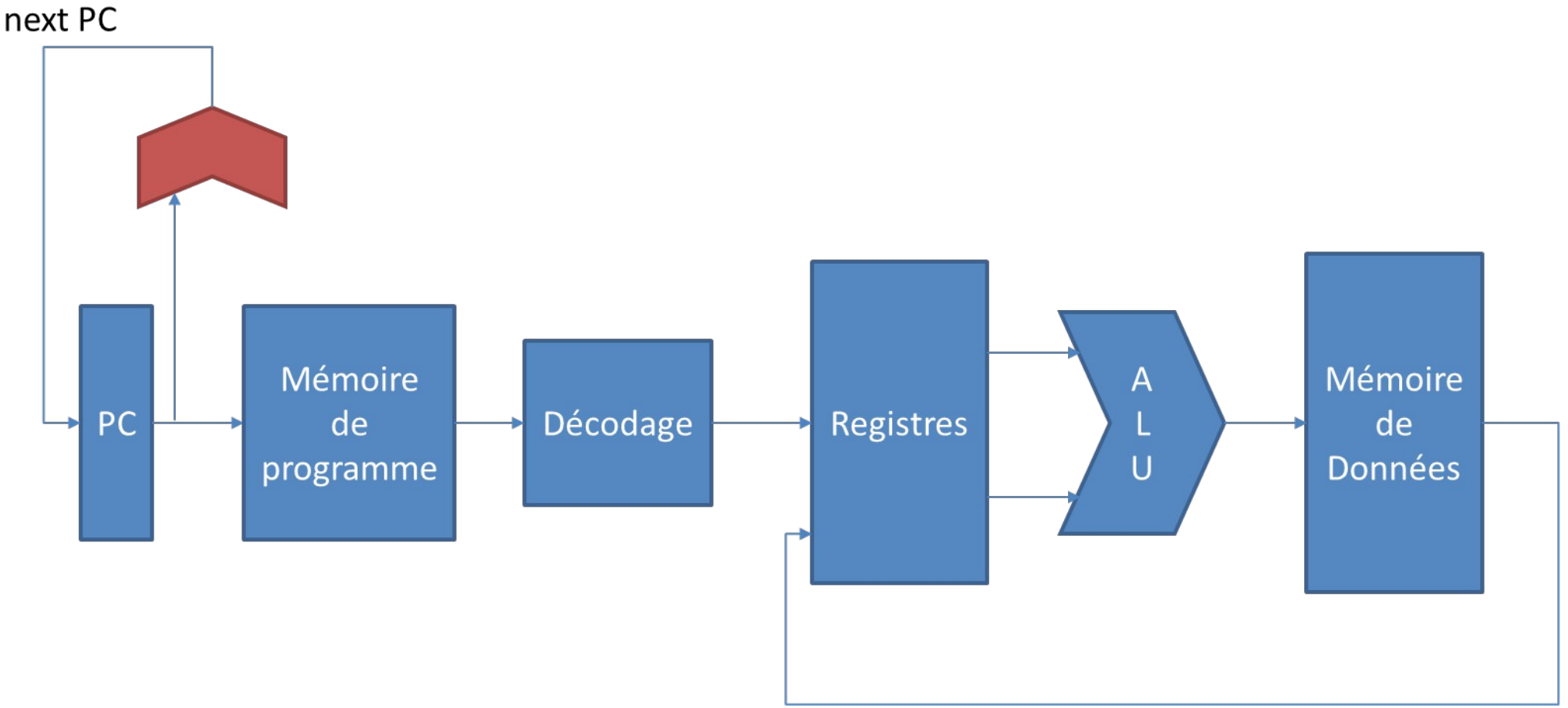


Illustration d'un fonctionnement pipeline

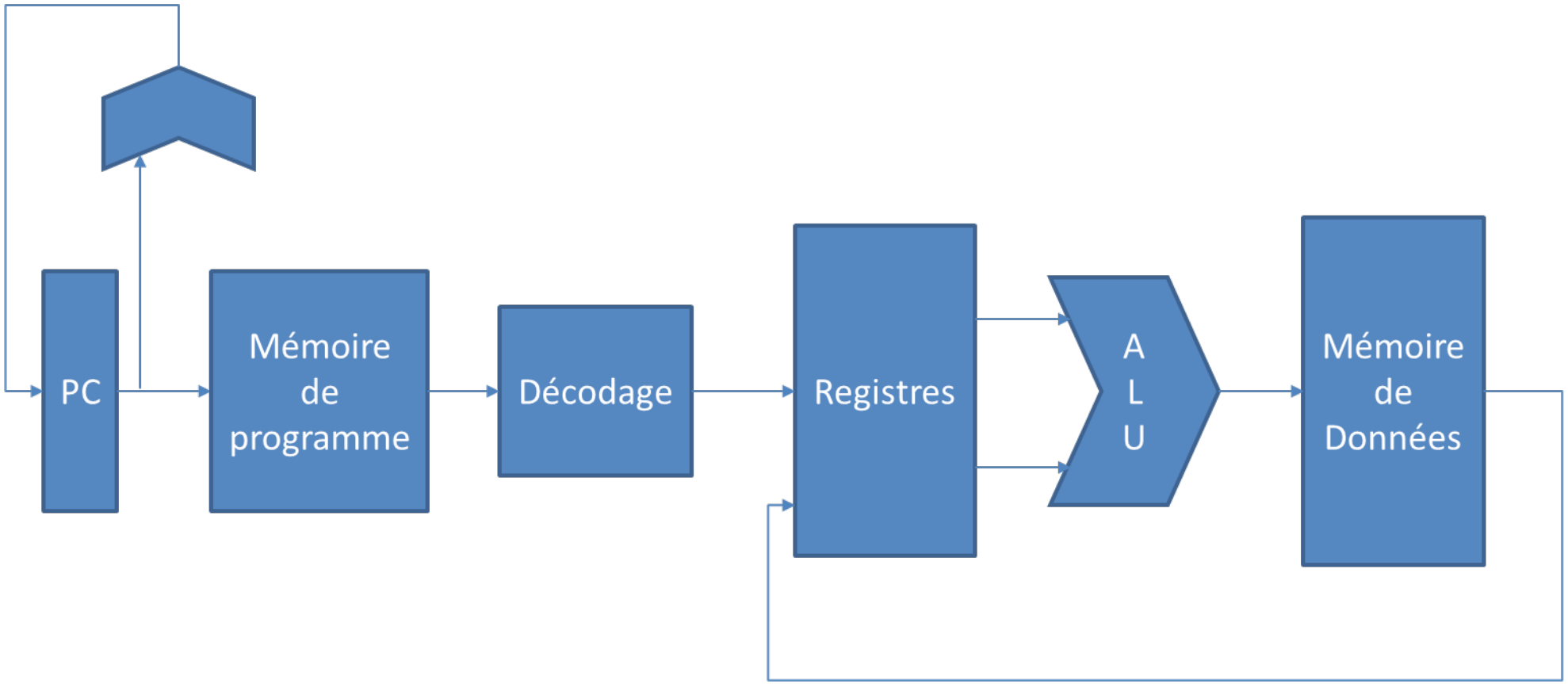


Illustration d'un fonctionnement pipeline

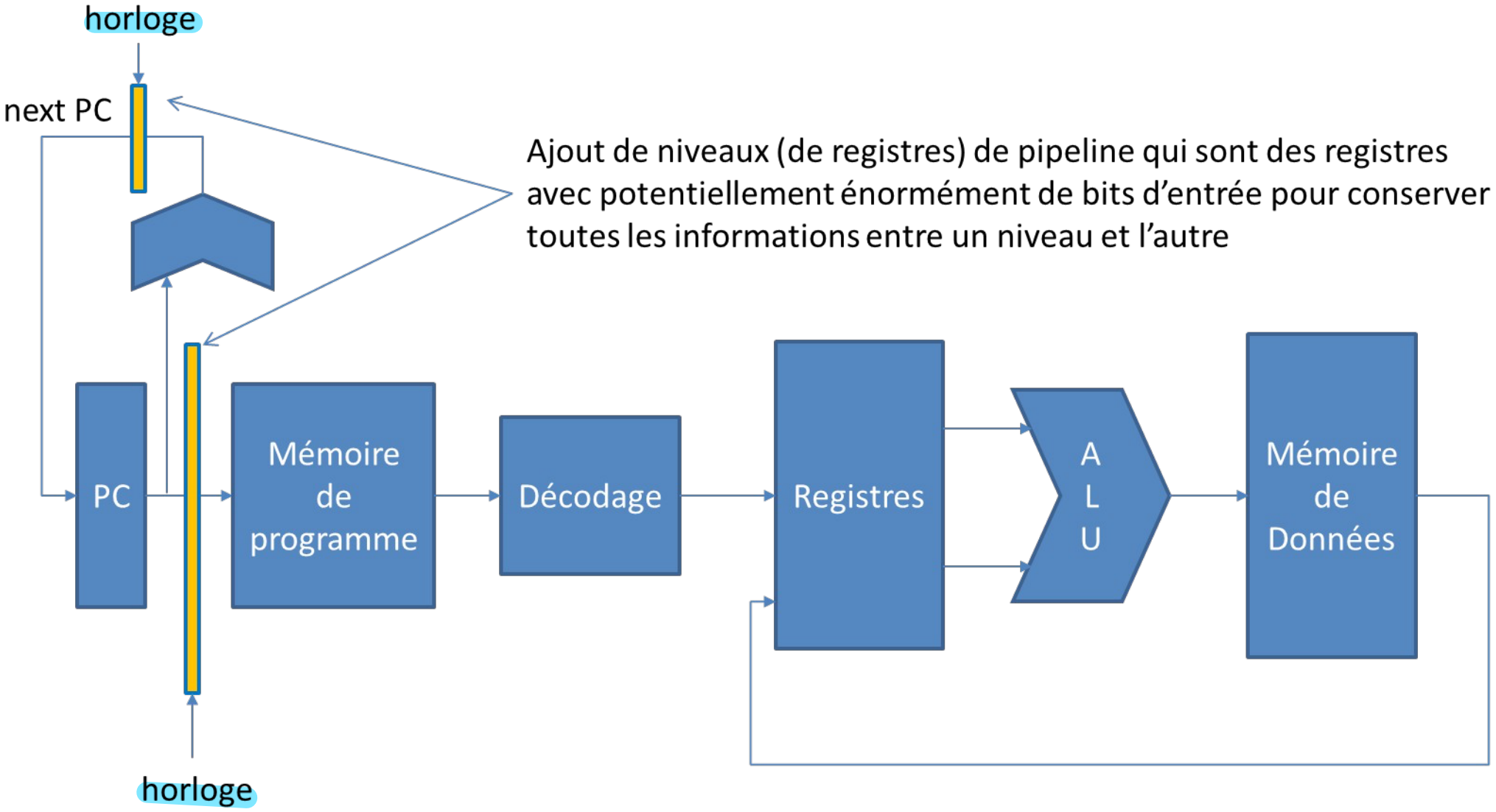


Illustration d'un fonctionnement pipeline

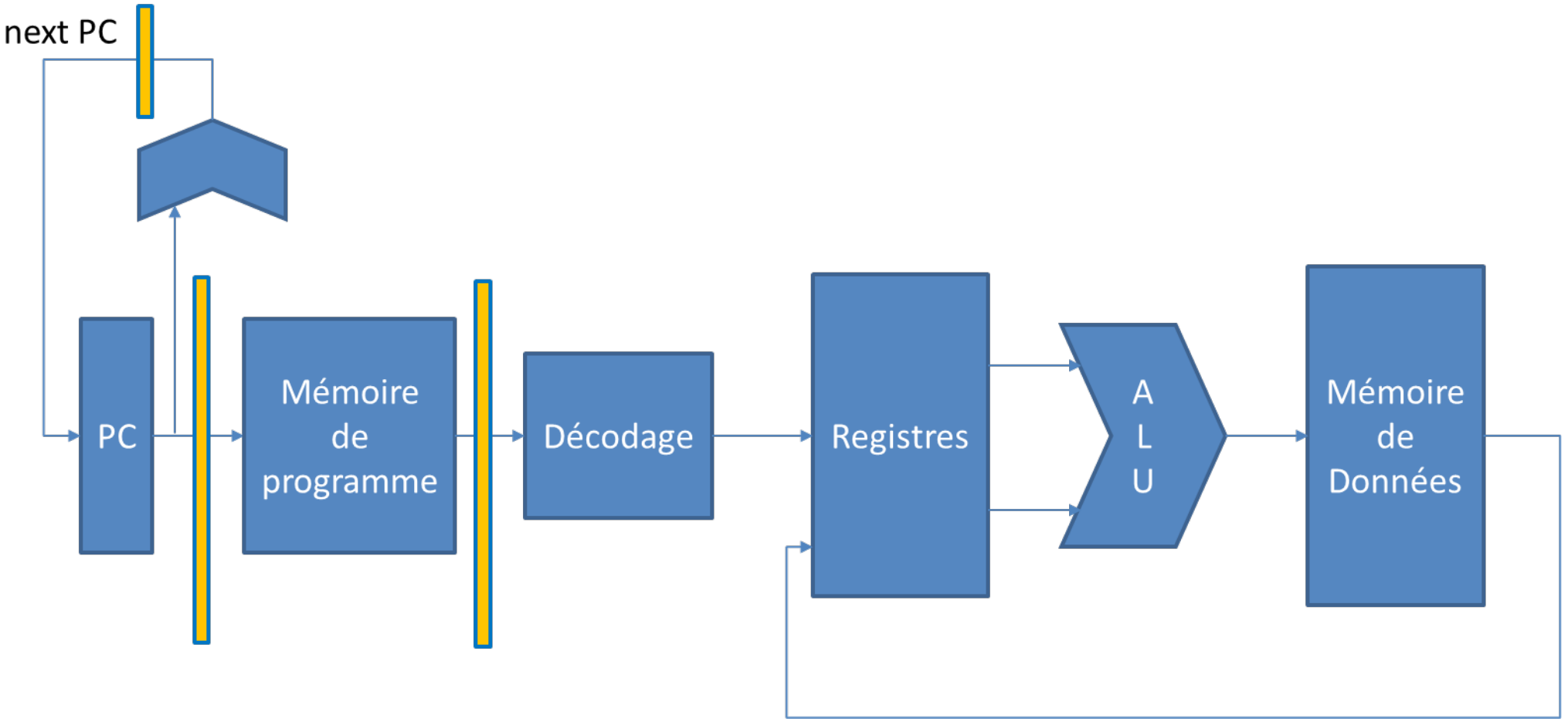


Illustration d'un fonctionnement pipeline

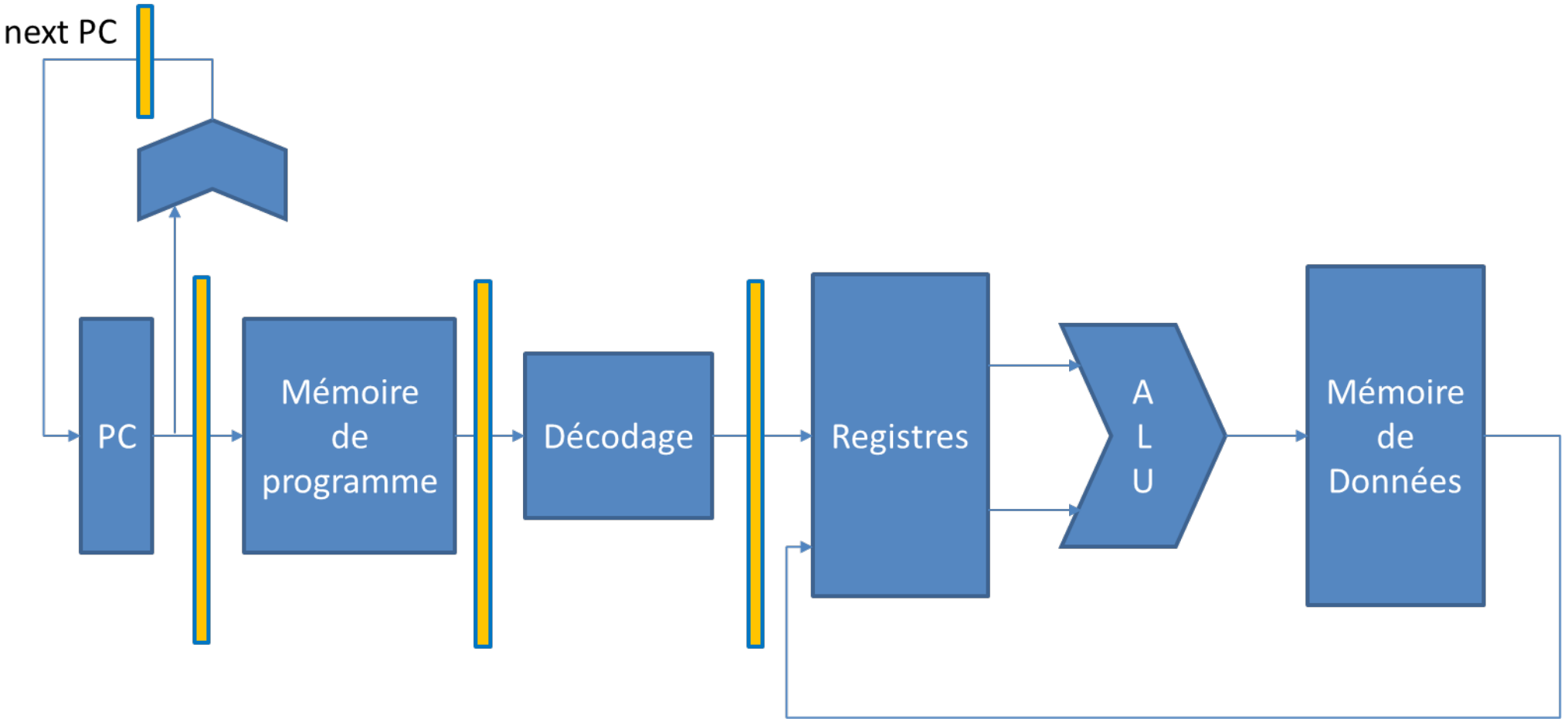


Illustration d'un fonctionnement pipeline

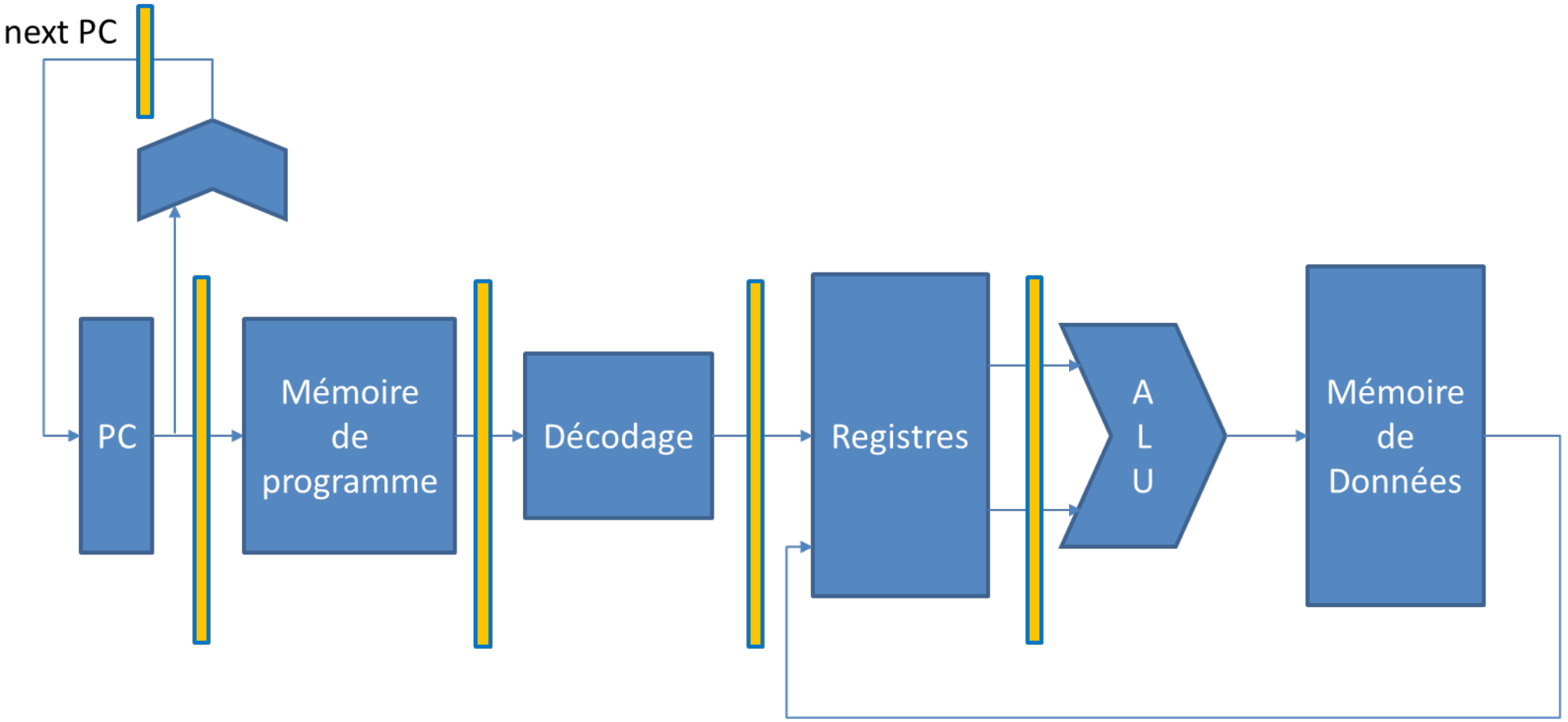


Illustration d'un fonctionnement pipeline

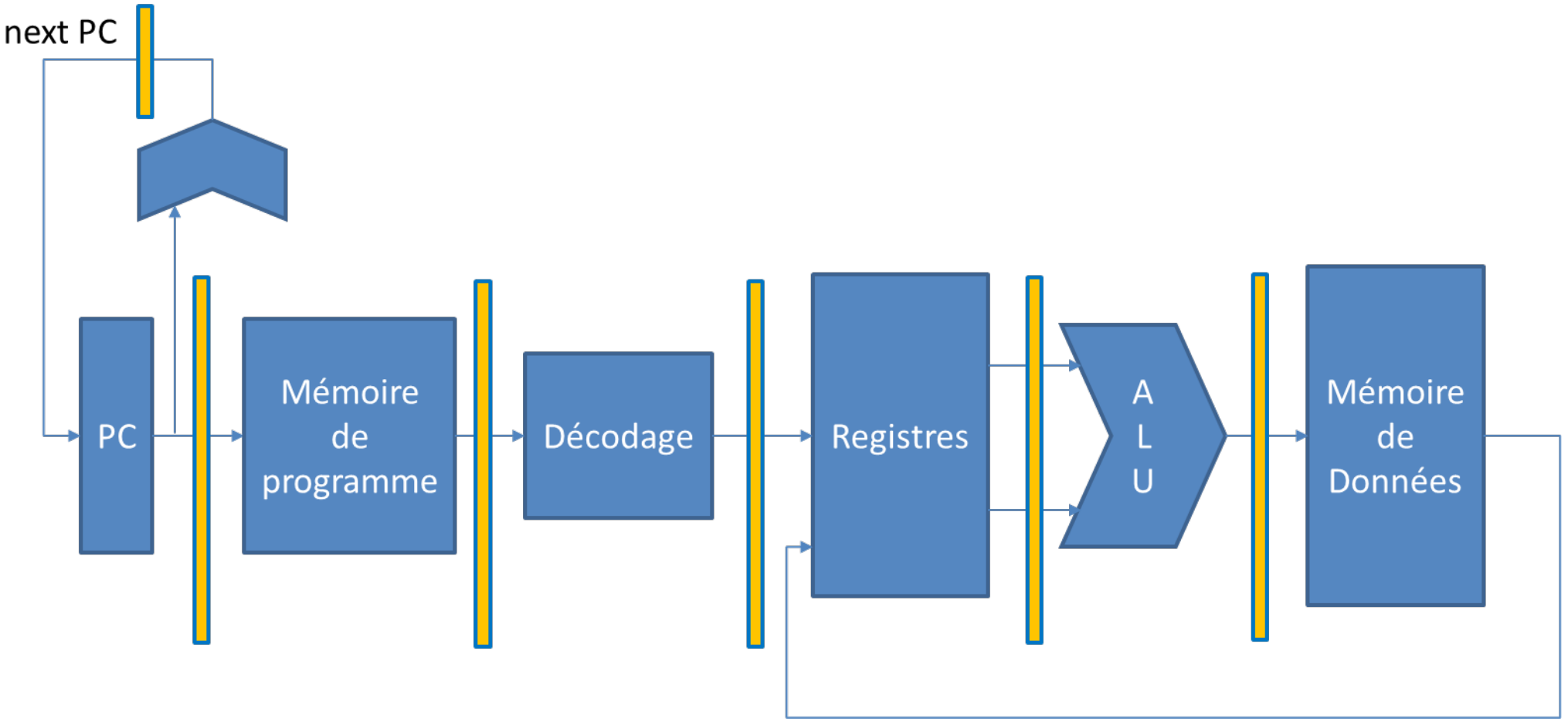
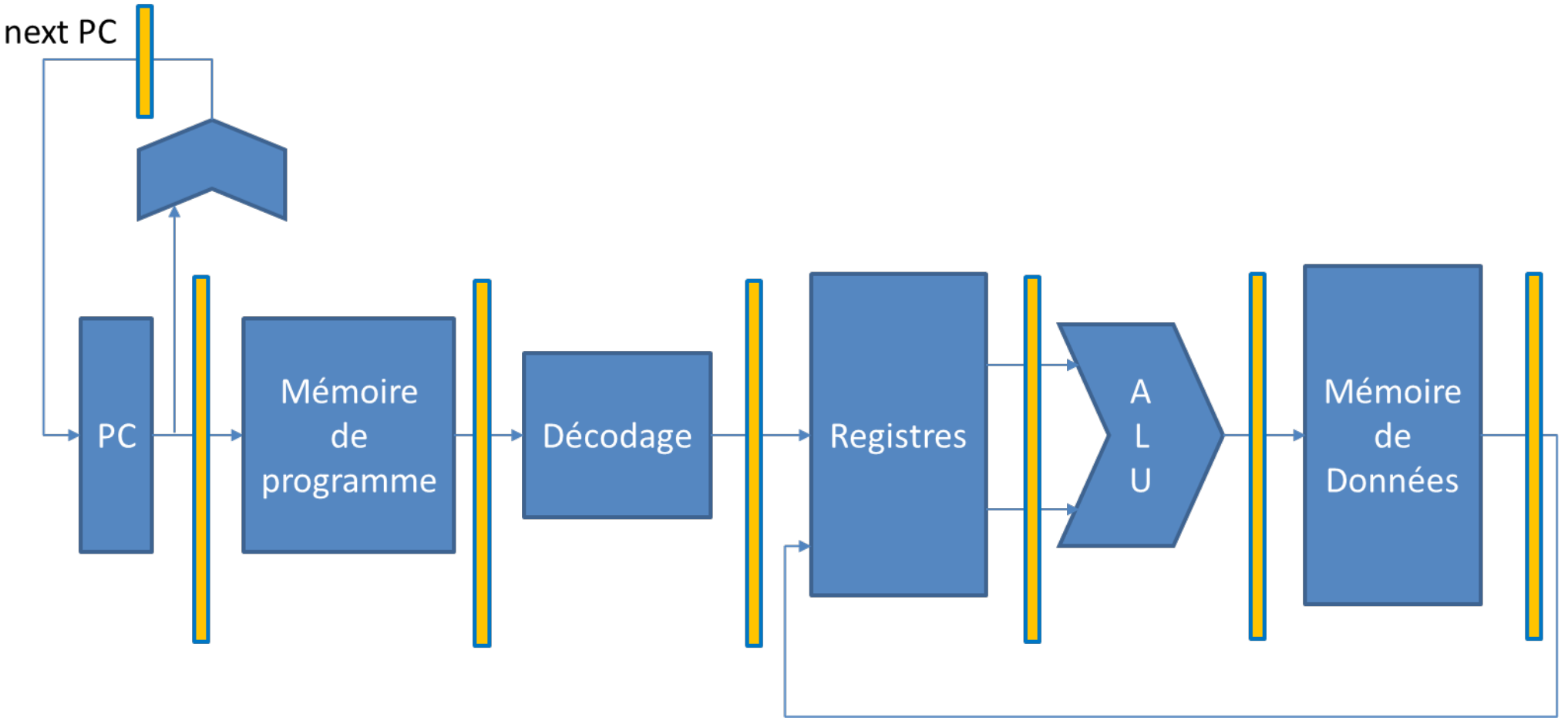
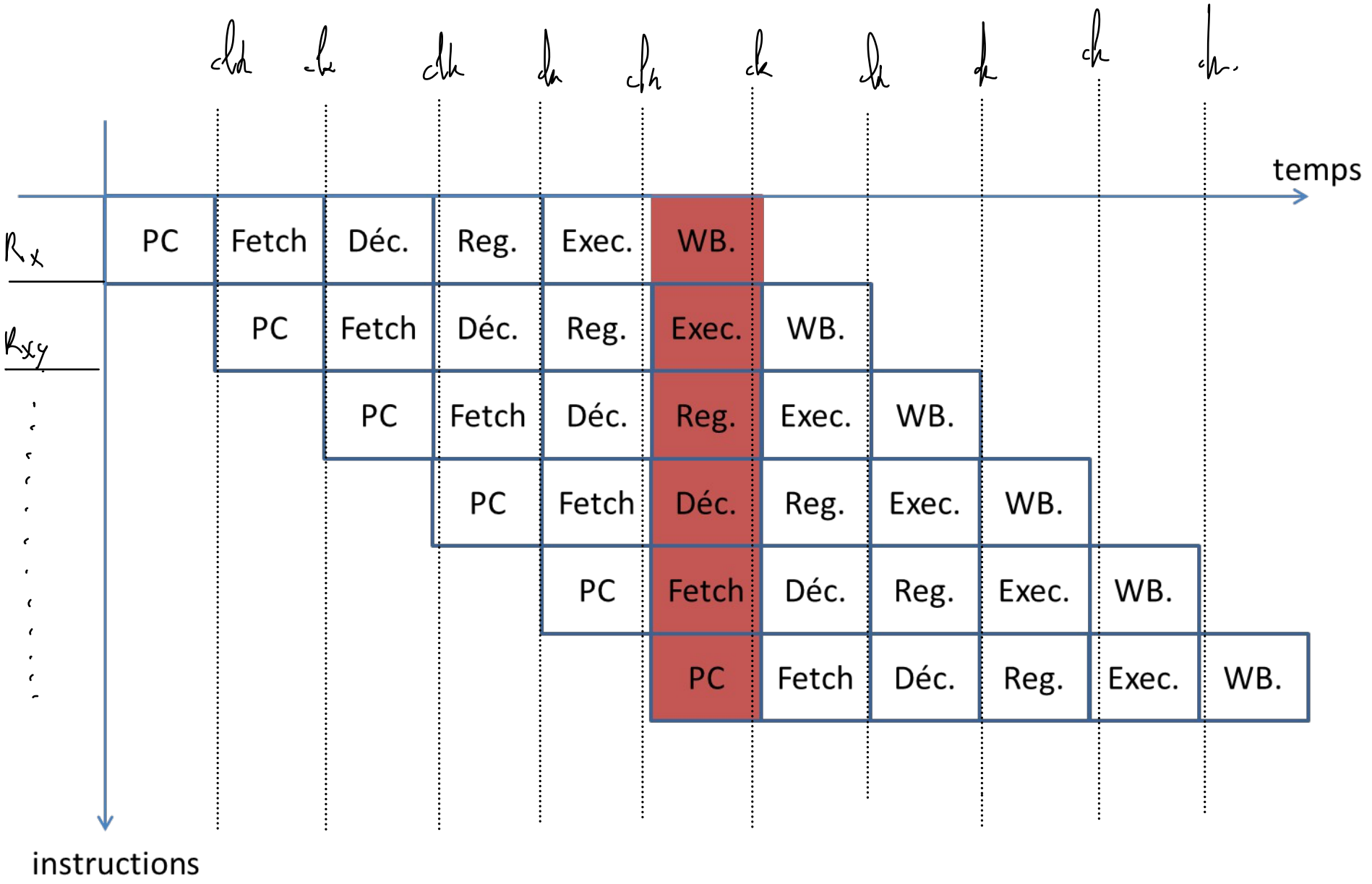


Illustration d'un fonctionnement pipeline




temps
Rx
Rxy
PC
Fetch
Déc.
Reg.
Exec.
WB.
instructions

1. Les Microcontrôleurs AVR

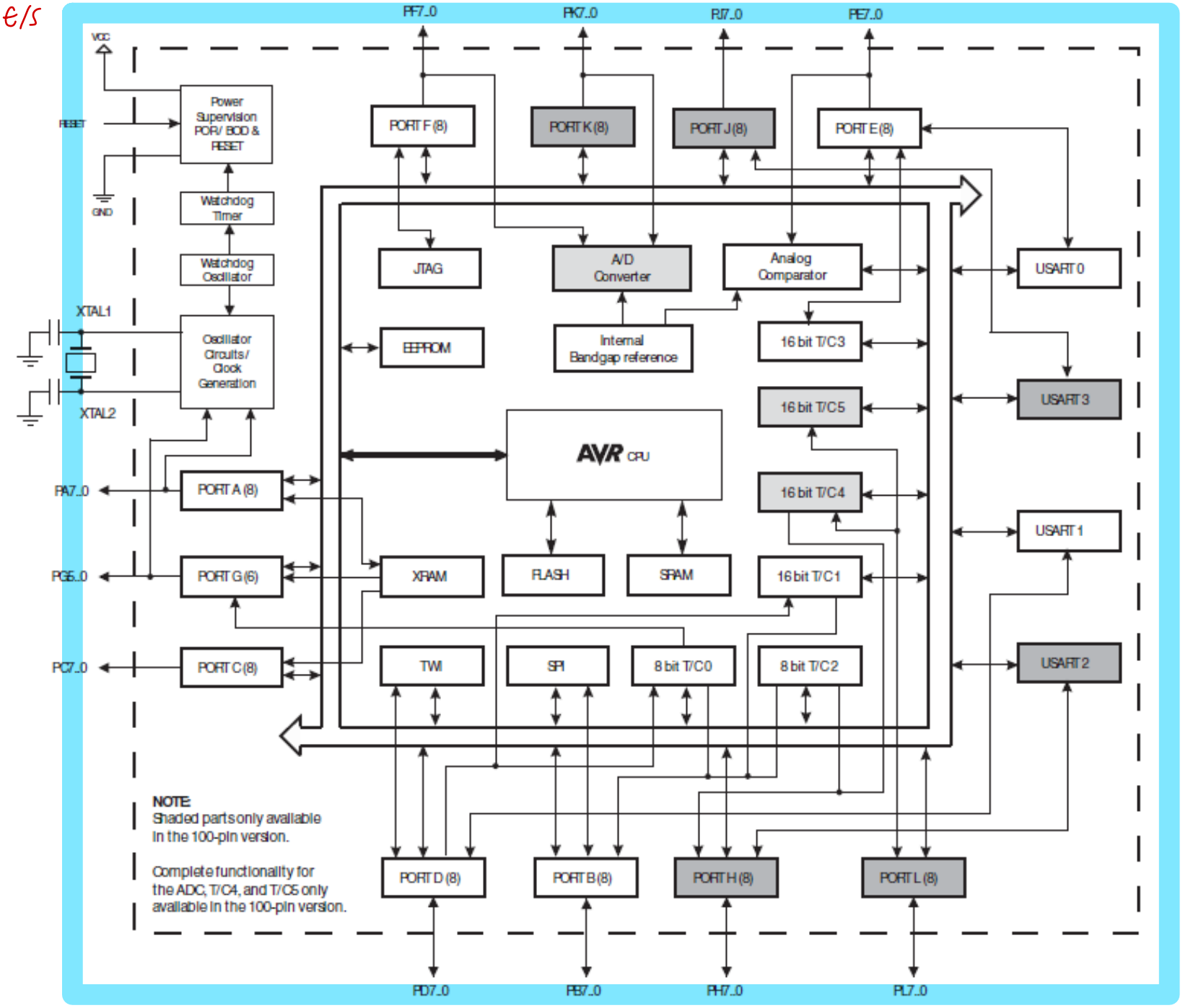
- **RISC** (Reduced Instruction Set Computer) est une structure interne d'un processeur basé sur un langage assembleur comportant un nombre minimum d'instructions simples
- Les premiers microprocesseurs étaient basés sur une conception interne de type **CISC** (abréviation de Complex Instruction Set Computer) incluant des instructions complexes capables d'exécuter en une opération plusieurs instructions simples
- En pratique, elles étaient peu utilisées et nécessitaient une conception complexe du cœur du microprocesseur, ralentissant l'exécution de l'ensemble des instructions (y compris des simples)
- L'avantage du RISC est un câblage direct des instructions à l'intérieur du circuit électronique, ce qui augmente la vitesse de traitement

2. Les Microcontrôleurs de type ATmega2560

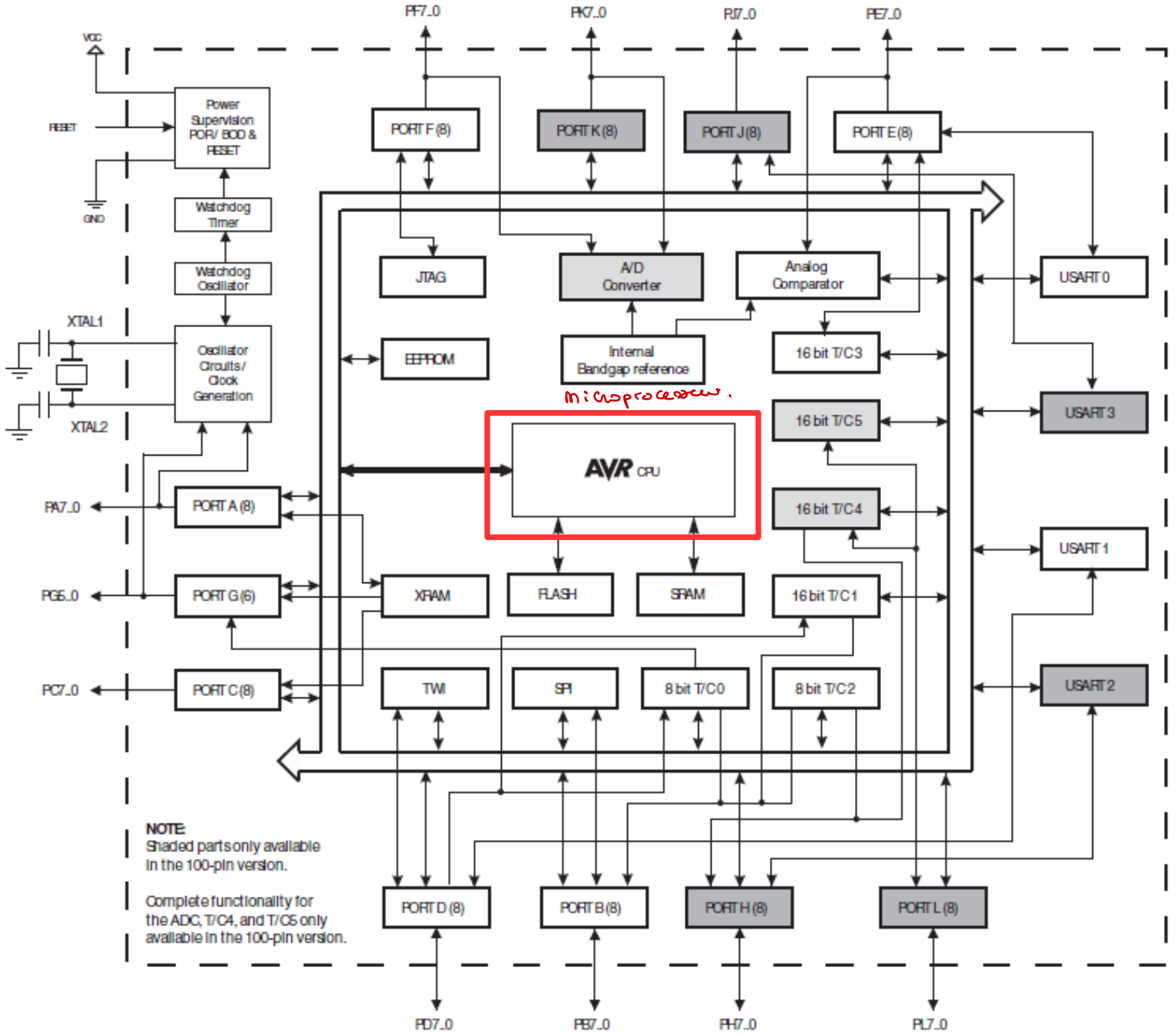
- Micro-contrôleur 8 bits de type RISC (Reduced Instruction Set Computer) à registres généraux
- 135 instructions, 32 registres 8 bits, cadence maximale 16 MIPS (millions d'instructions par seconde), multiplieur câblé 2 cycles
- Nombreux périphériques dont :
  - Timers/Counters 8 bits, 16 bits,
  - 4 canaux PWM, 12 canaux PWM avec résolution de 16 bits,
  - 16 canaux ADC 10 bits,
  - 4 canaux programmable UART,
  - un WathDog programmable...
- 86 entrées/sorties programmables

Architecture interne du ATmega2560

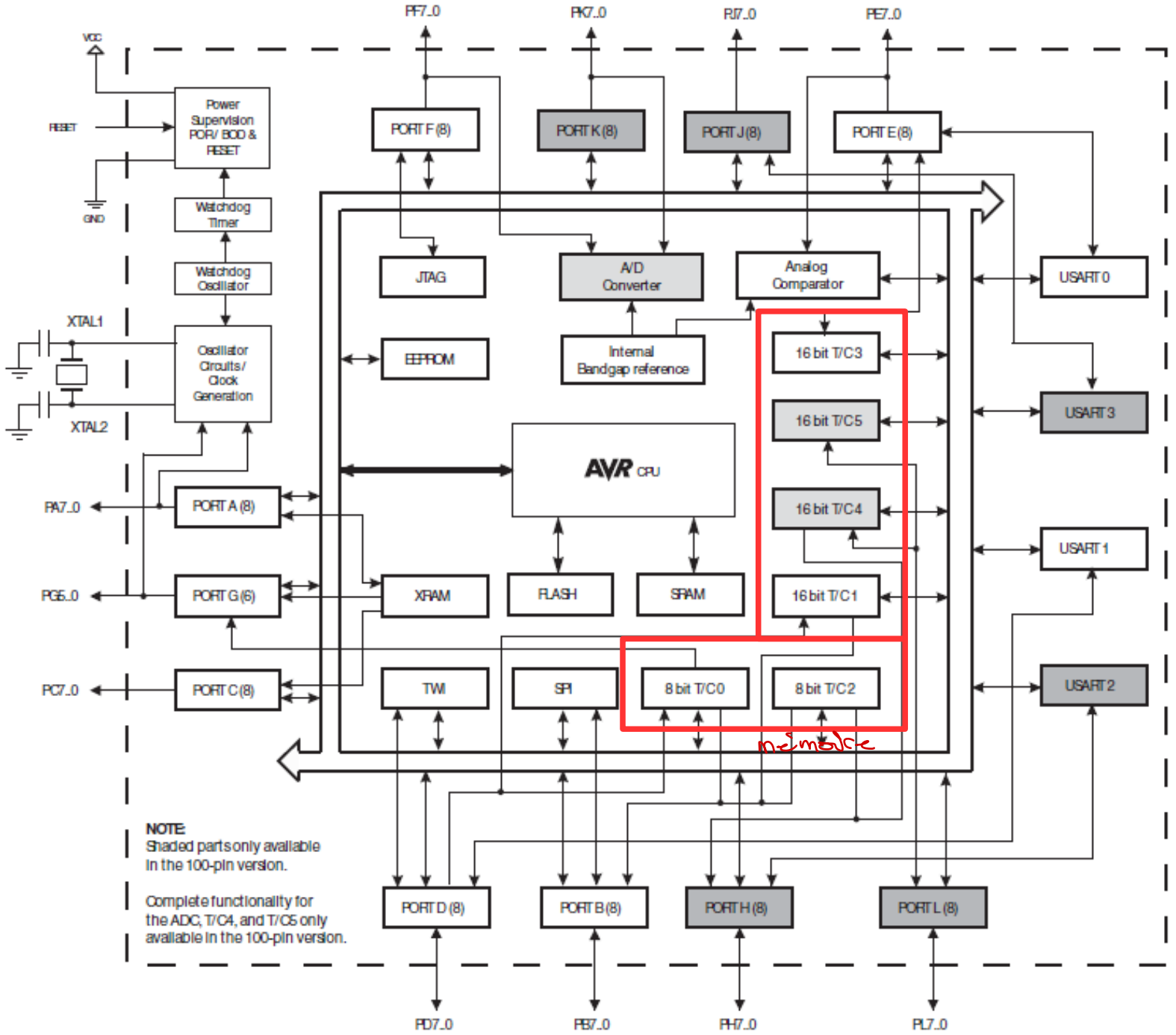
Port E/S



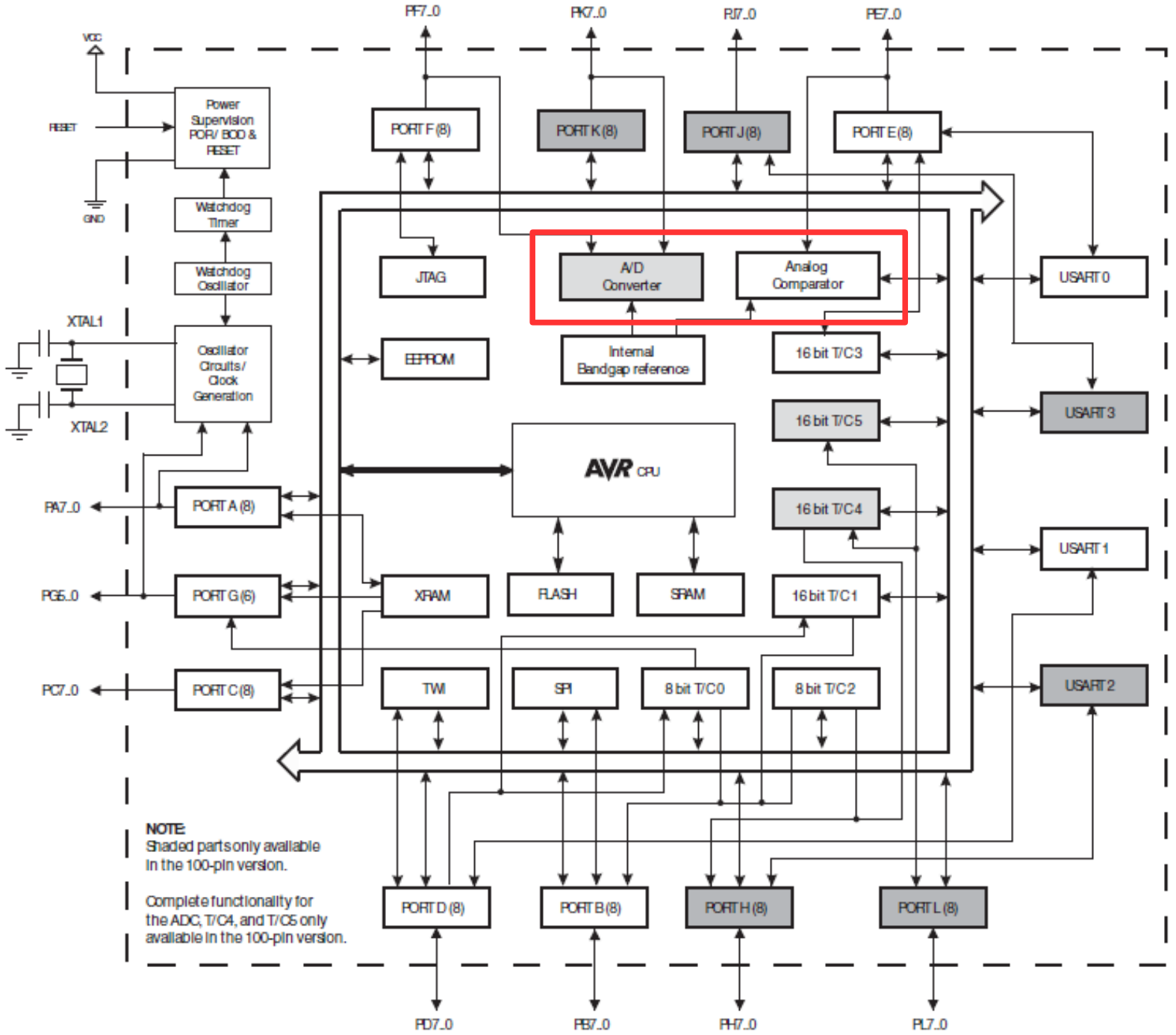
Architecture interne du ATmega2560



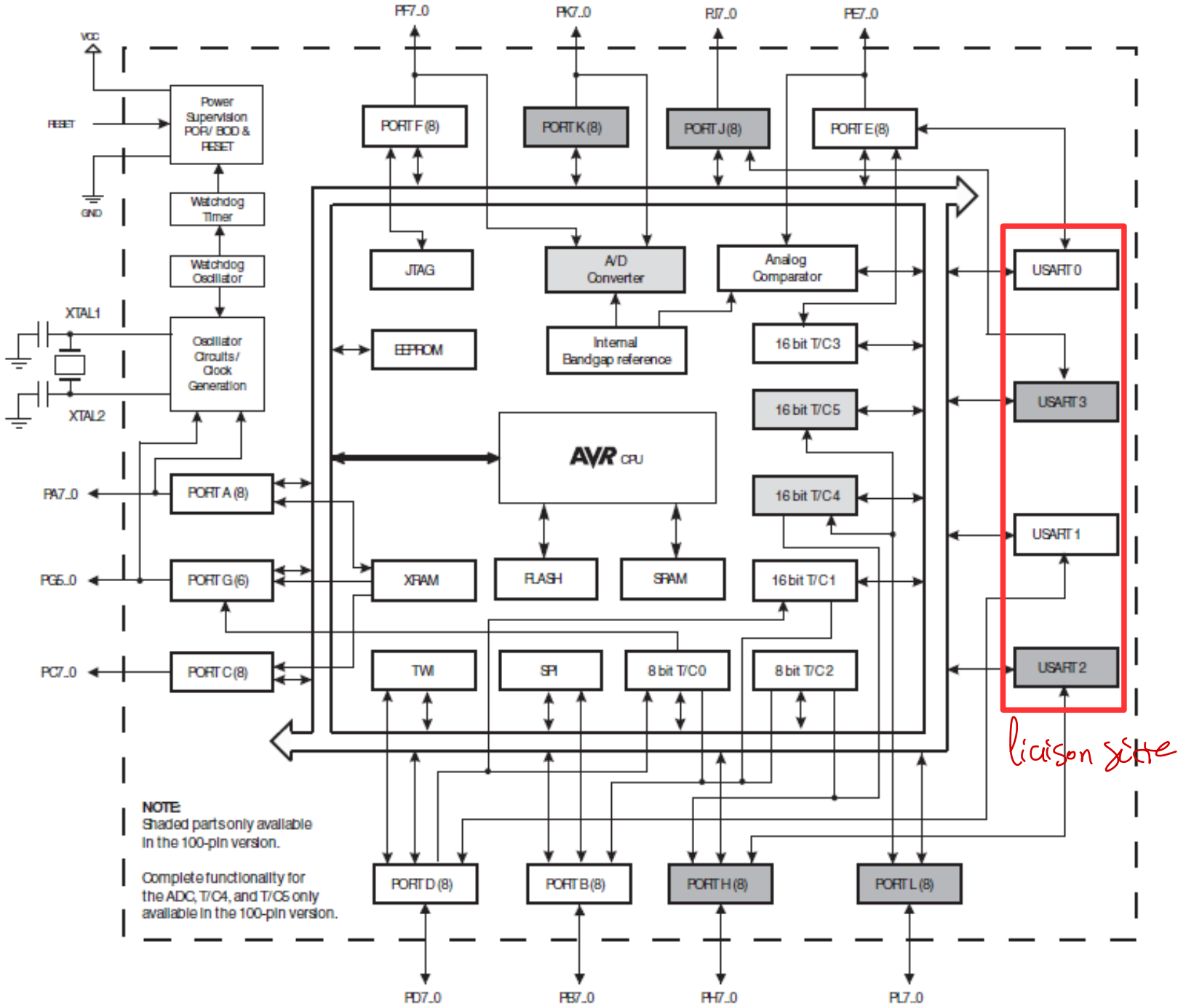
Architecture interne du ATmega2560



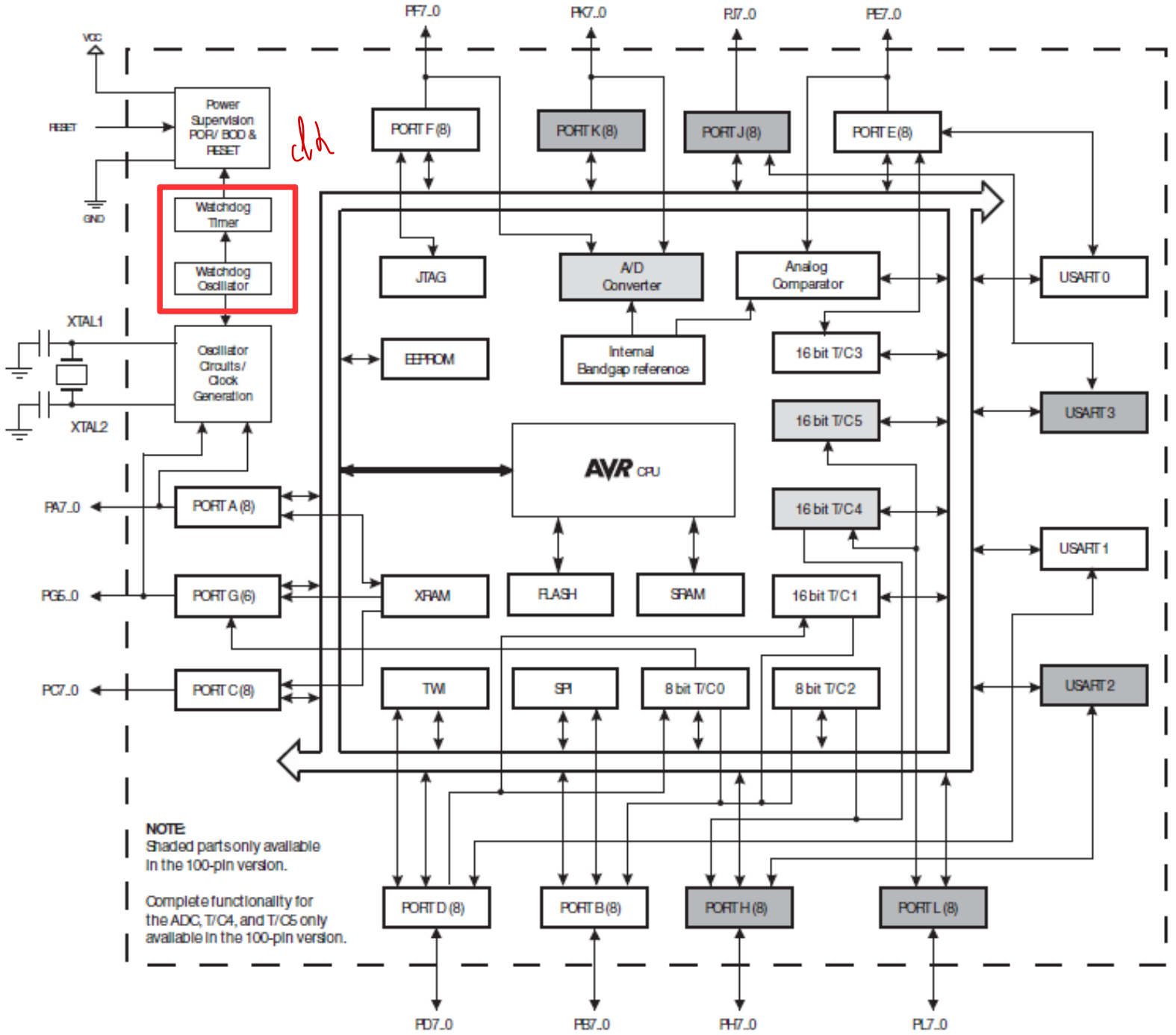
Architecture interne du ATmega2560



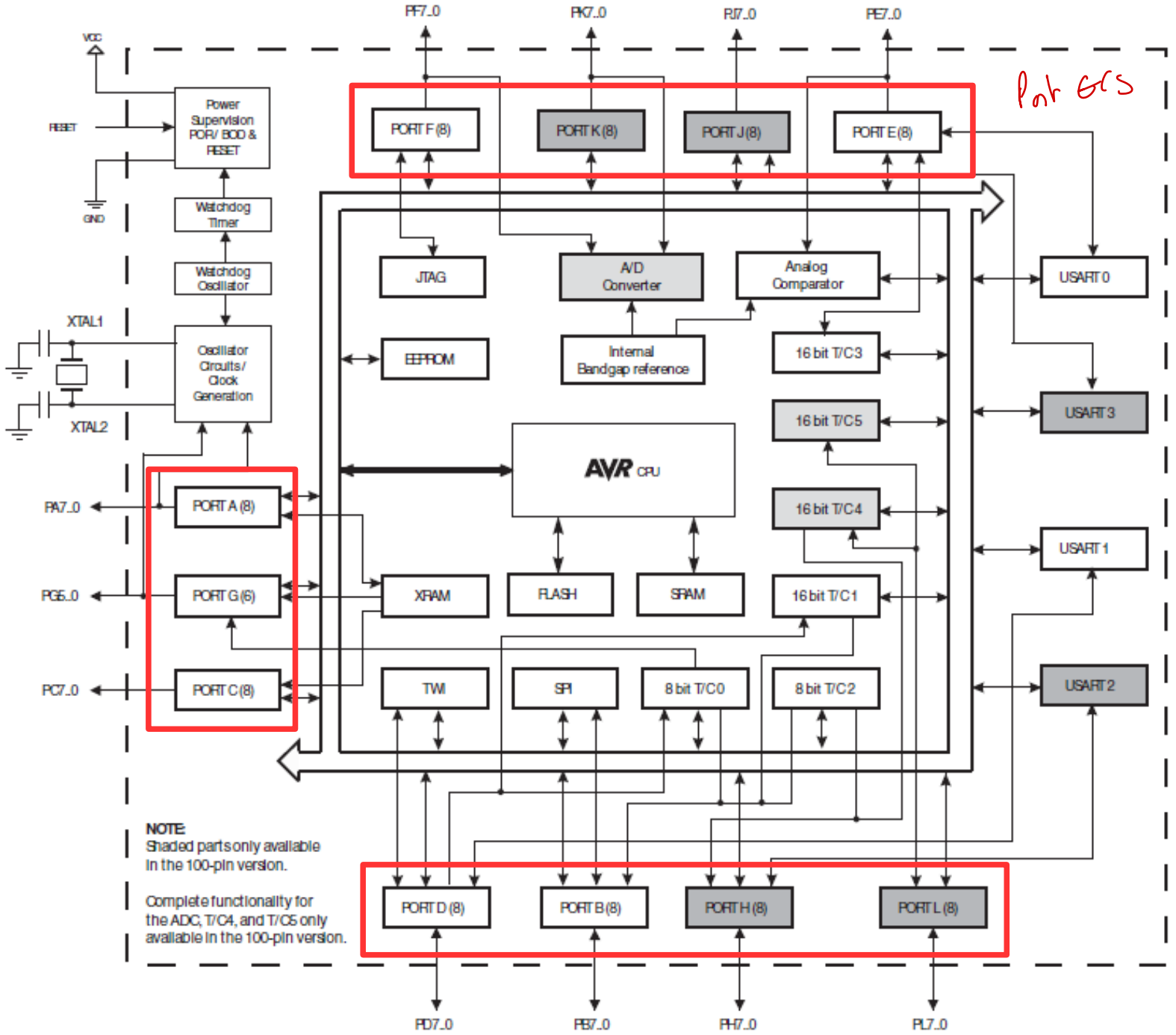
Architecture interne du ATmega2560



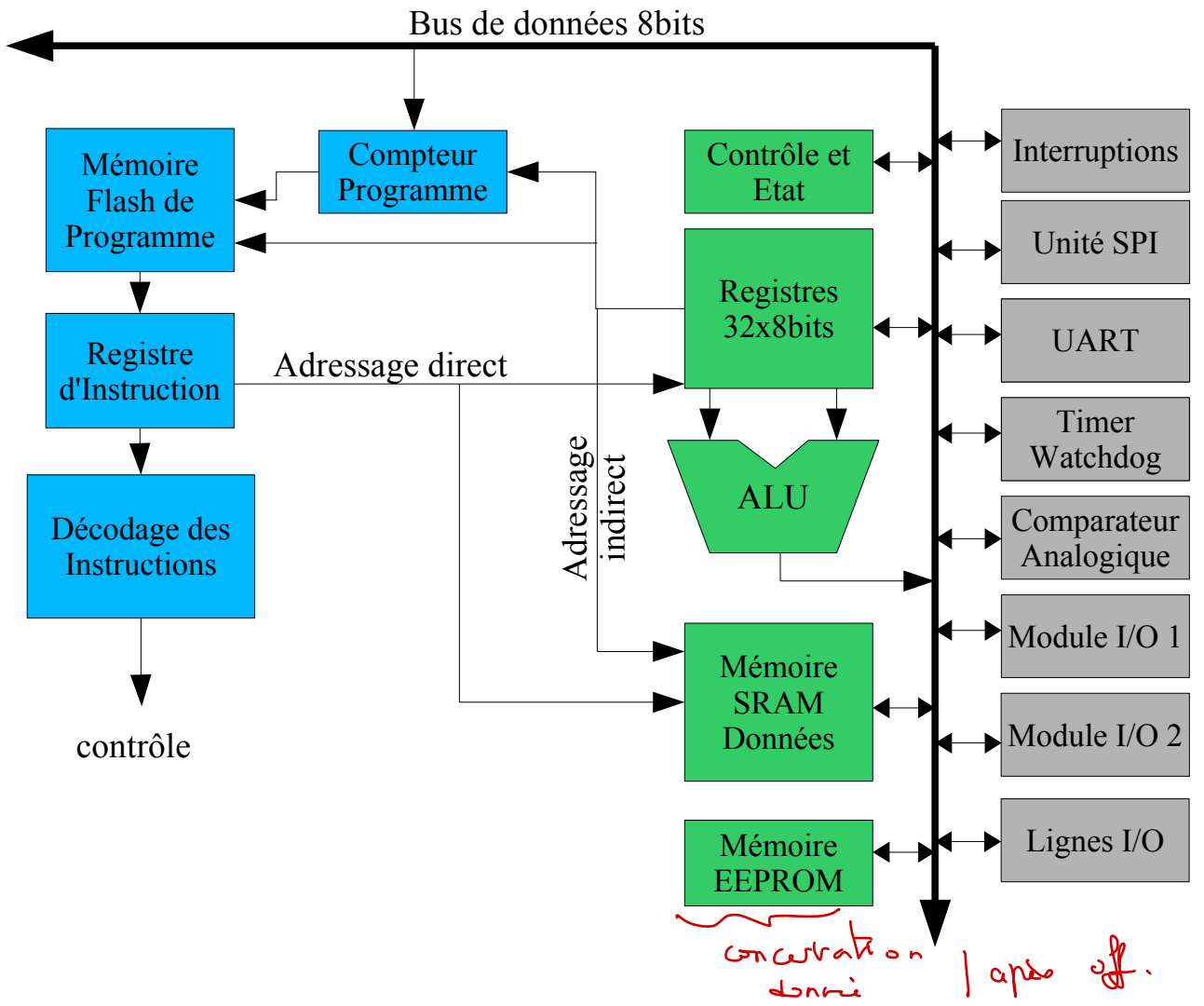
Architecture interne du ATmega2560



Architecture interne du ATmega2560



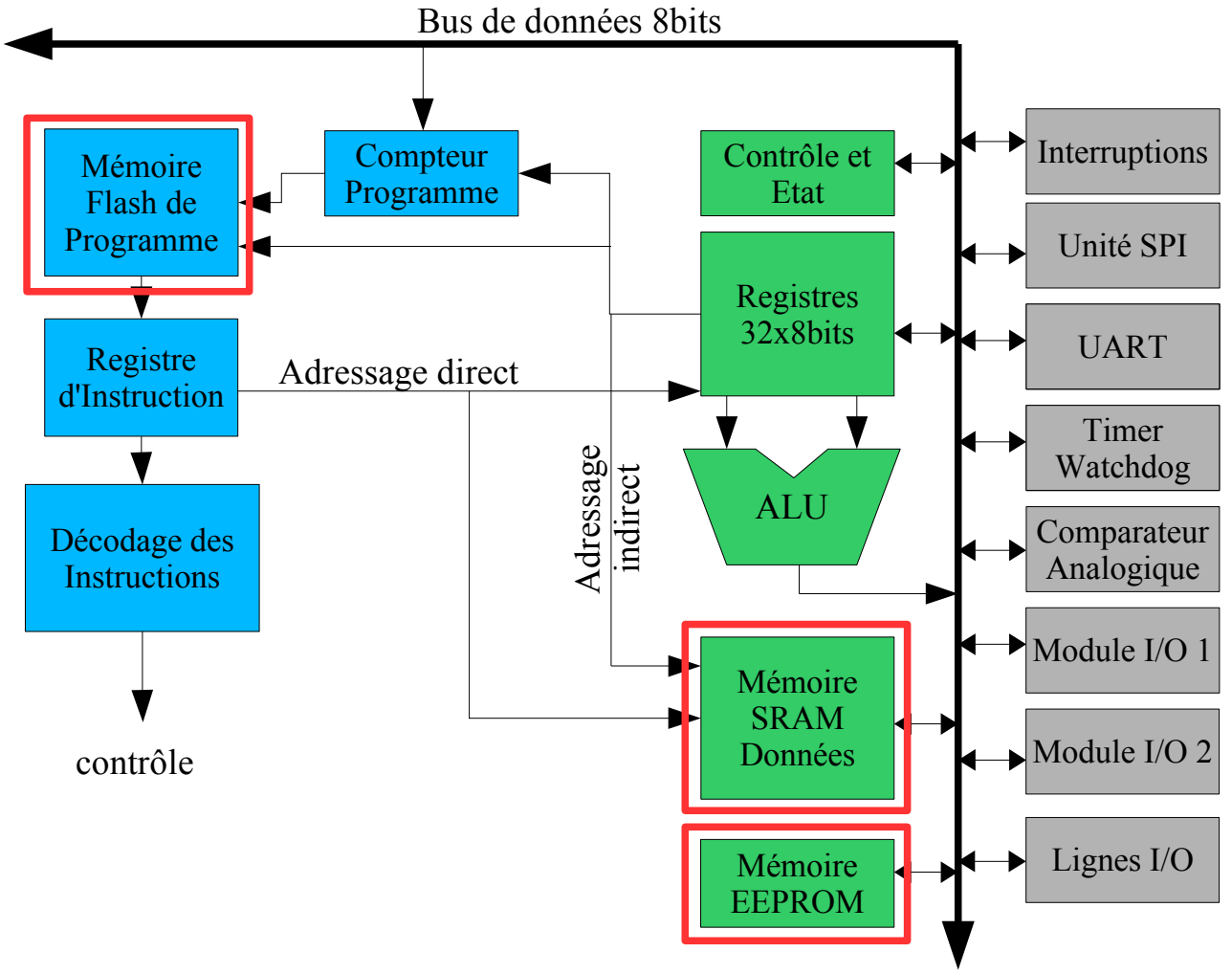
Architecture interne du ATmega2560



Architecture interne du ATmega2560

Architecture HARVARD

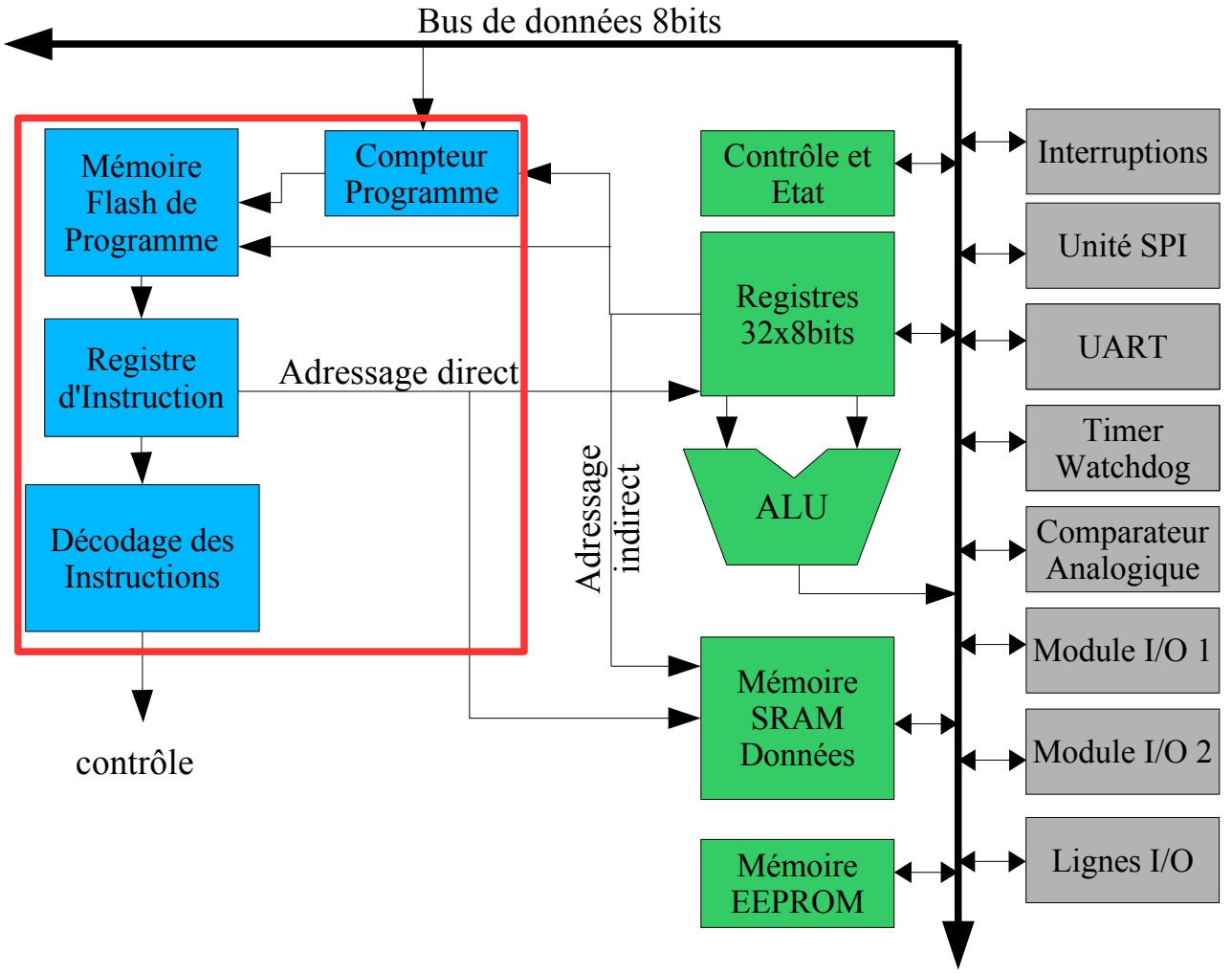
- Les bus d'instruction (16 bits) et de données (8 bits) sont séparés
- 256 Ko de mémoire Flash de programme
- 8 Ko de mémoire SRAM de données
- 4 Ko de mémoire EPROM de données



Architecture interne du ATmega2560

Architecture HARVARD

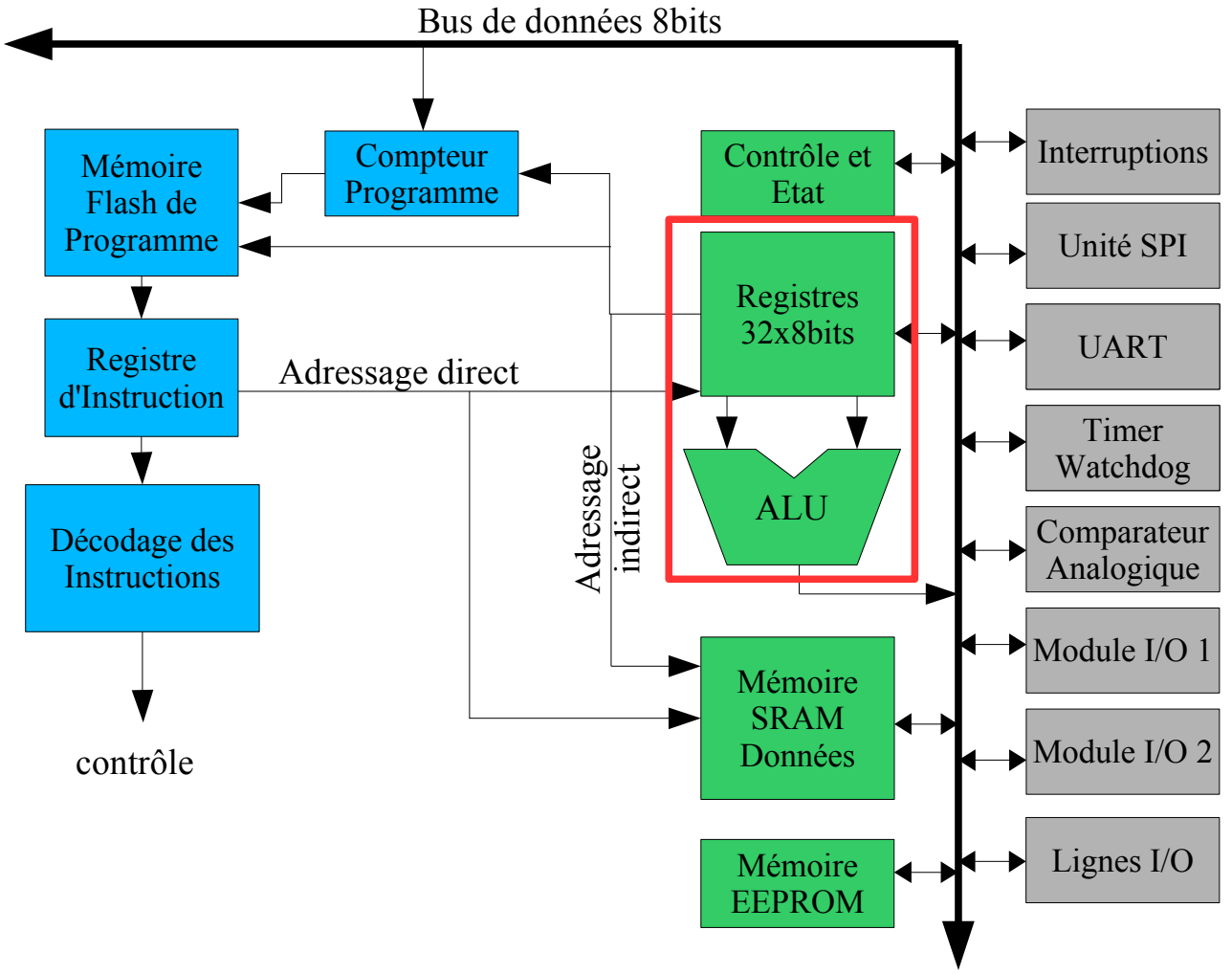
- Bloc d'accès aux instructions (Fetch and Decode)
- 1 temps de cycle pour quasiment toutes les instructions
- Instructions sur 16 bits



Architecture interne du ATmega2560

Processeur AVR

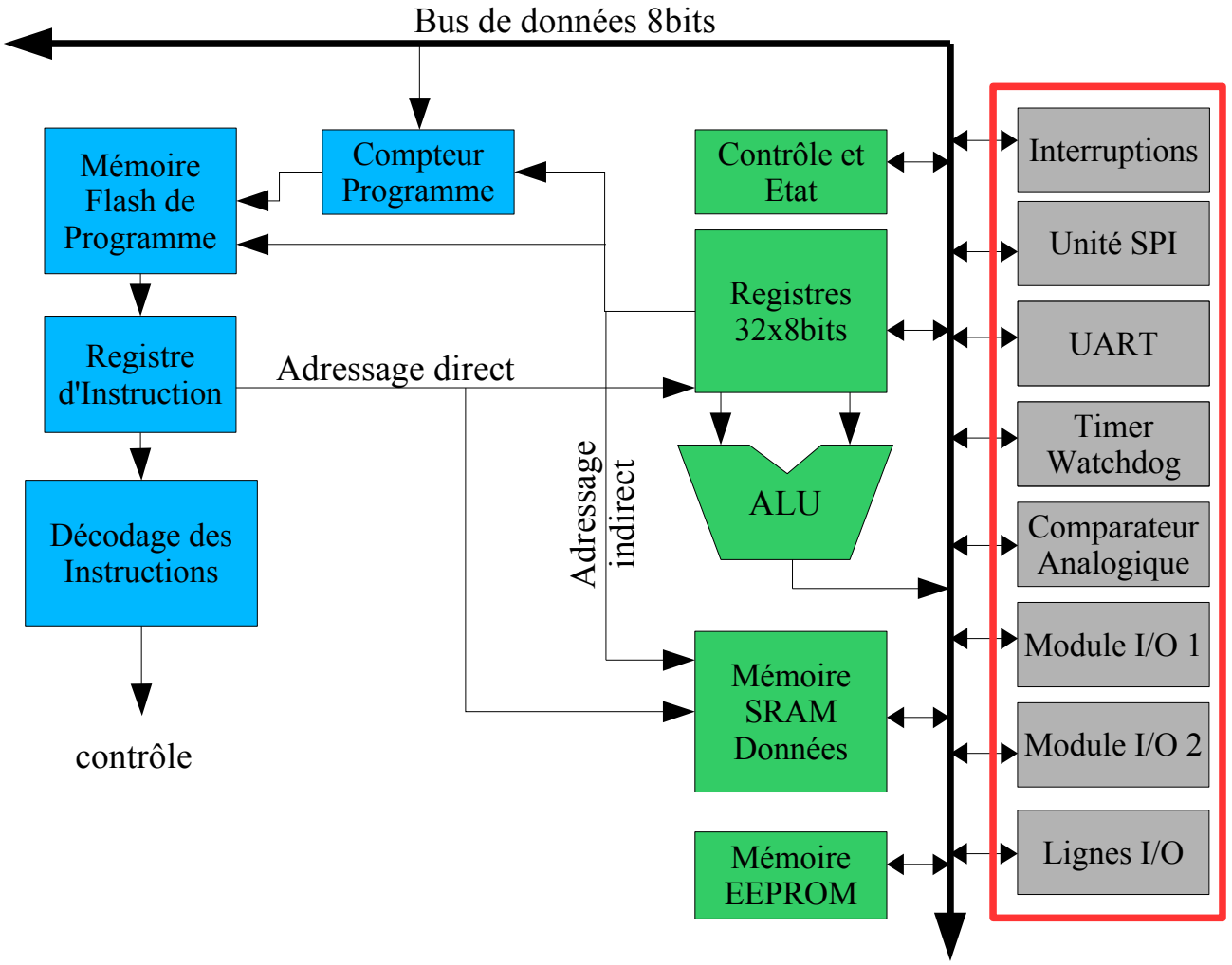
- Instructions de type ALU
- Multiplieur câblé 2 cycles
- Opérations sur les bits individuels
- 32 registres généraux



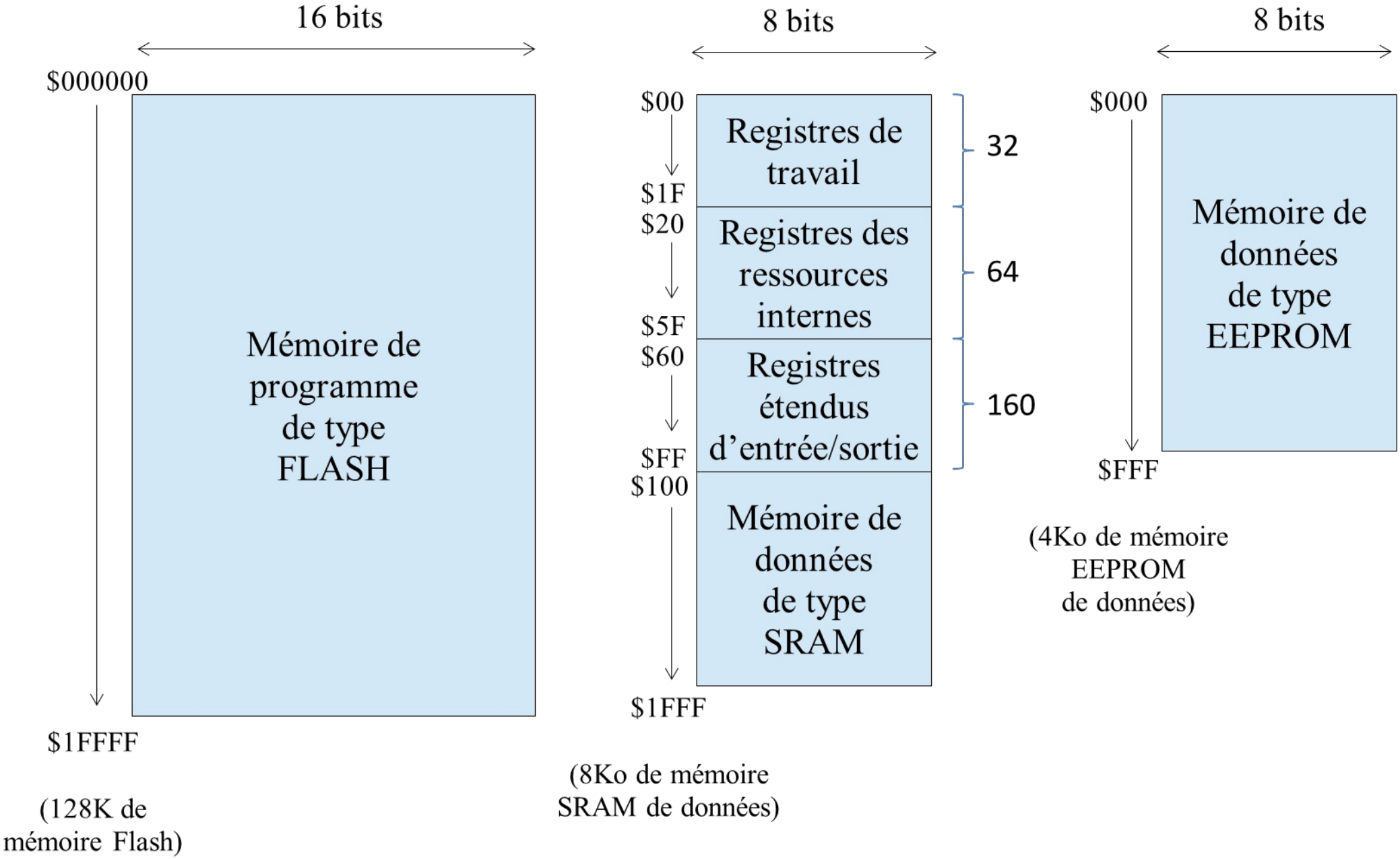
Architecture interne du ATmega2560

Processeur AVR

- Fonctions pour l'accès aux lignes d'entrée/sortie
- Fonctions spécifiques pour la gestion des interruptions



Les accès aux mémoires de l'AVR ATmega2560



3. Les ressources internes du CPU AVR

1) L'Unité Arithmétique et Logique (ALU)
2) Le registre d'état
3) Le registre de pile
4) Le registre RAMPZ
5) Le registre EIND
6) La file des 32 registres généraux
7) Le registre d'état spécial
8) Les cycles du processeur ATmega2560

1) l'ALU

- L'ALU haute performances de l'AVR peut fonctionner avec n'importe lequel des 32 registres 8 bits.
- En un cycle d'horloge, l'ALU peut exécuter n'importe quelle opération entre les registres ou entre les registres et une valeur immédiate
- Les opération de l'ALU sont séparées en trois catégories : les opérations arithmétiques, logiques et les opération au niveau des bits
- L'ALU est équipée d'un multiplieur câblé capable d'effectuer ses calculs en 2 cycles d'horloge

2) Le registre d'état

- Le registre d'état (SREG) contient des informations à propos de la dernière opération arithmétique exécutée
- Ces informations peuvent être utilisées pour modifier le cours d'un programme par l'exécution d'instructions de branchement conditionnel
- Le registre d'état n'est pas automatiquement stocké lorsque une interruption arrive. C'est au programmeur de le faire

SREG

Bits	7	6	5	4	3	2	1	0
0x5F	I	T	H	S	V	N	Z	C
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Valeur par défaut	0	0	0	0	0	0	0	0

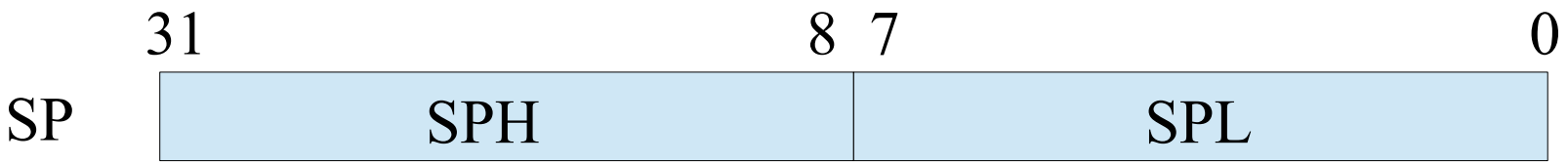
- Bit 7 – I : Global Interrupt Enable
- Bit 6 – T : Bit Copy Storage
- Bit 5 – H : Half Carry Flag
- Bit 4 – S : Signe
- Bit 3 – V : Overflow signé
- Bit 2 – N : Bit Negatif
- Bit 1 – Z : Zero
- Bit 0 – C : Carry

3) Le registre de pile

- Ce registre de pile (SP : Stack Register) est de 16 bits
- Il est automatiquement initialisé au cours d'une opération de Reset à la dernière adresse de la mémoire RAM (RAMEND : $1FFF)
- Le registre de pile est automatiquement décrémenté à chaque fois qu'une nouvelle valeur est empilée et incrémenté à chaque fois qu'une valeur est dépilée
- Les instructions qui modifient la pile sont :
  - PUSH, CALL, RCALL, ICALL, EICALL
  - POP, RET, RETI
- La pile est modifiée de 1 à chaque instruction PUSH et POP et de 3 à chaque CALL, RCALL, ICALL, EICALL, RET et RETI (à cause de la taille du compteur programme)

		15	14	13	12	11	10	9	8
SPH	0x5E	SP16	SP15	SP14	SP13	SP12	SP11	SP9	SP8
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND

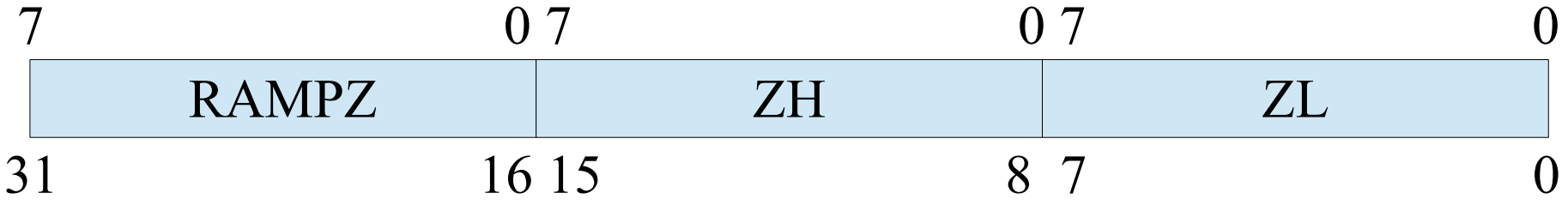
		7	6	5	4	3	2	1	0
SPL	0x5D	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0
	Bits	7	6	5	4	3	2	1	0
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND	RAMEND



4) Le registre d'adresse RAMPZ

- Pour l'exécution des instructions ELPM ou SPM (accès à la mémoire de programme), le registre d'index Z est concaténé avec le registre RAMPZ pour former une adresse complète sur 18 bits. Les bits inutilisés sont toujours lus comme des zéros.

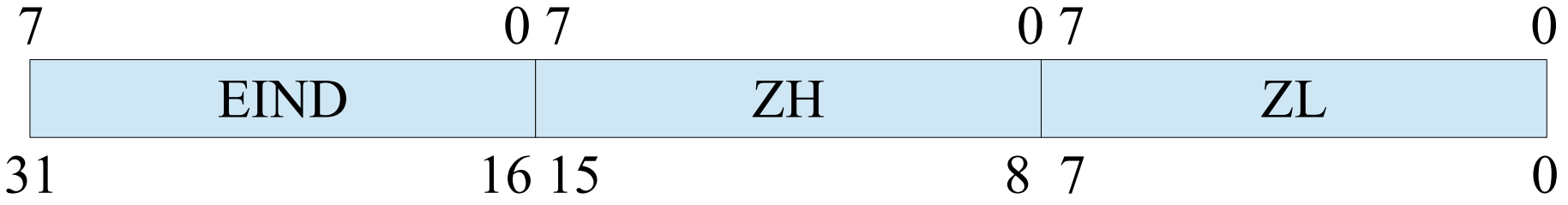
	Bits	7	6	5	4	3	2	1	0
RAMPZ	0x5B	-	-	-	-	-	-	RZ1	RZ0
	Read/Write	R	R	R	R	R	R	R/W	R/W
	Valeur par défaut	0	0	0	0	0	0	-	-



5) Le registre d'adresse EIND

- Pour l'exécution des instructions EICALL ou EIJMP (branchements), le pointeur indirect vers la routine est une concaténation des registres EIND et Z. Là encore, les bits inutilisés sont lus comme des zéros.

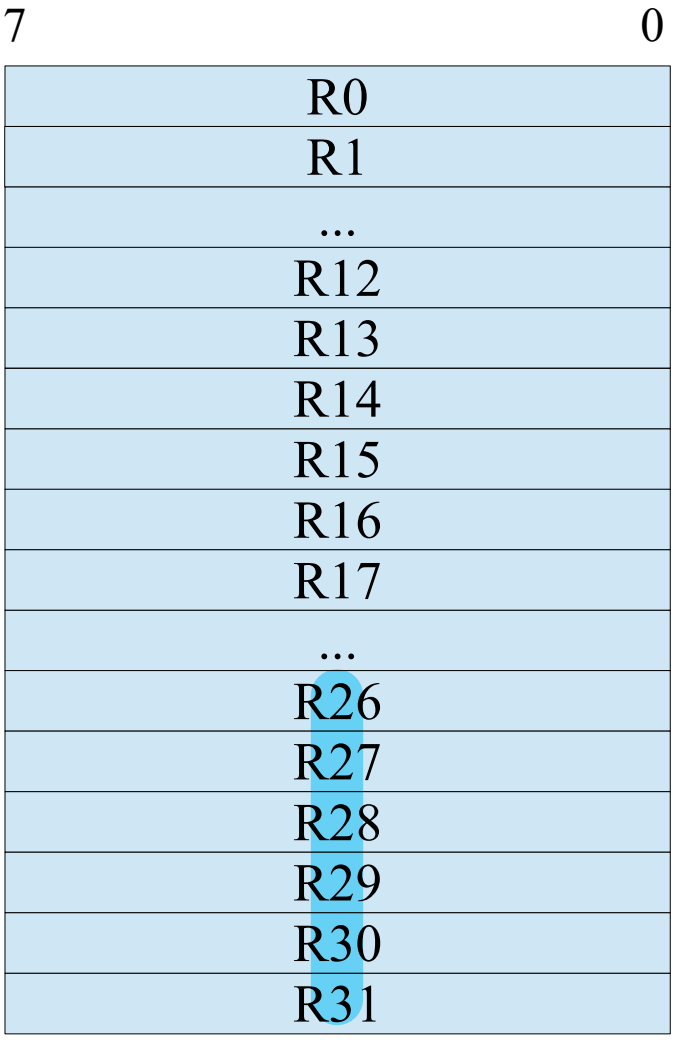
Bits	7	6	5	4	3	2	1	0
EIND 0x5C	-	-	-	-	-	-	-	EID0
Read/Write	R	R	R	R	R	R	R	R/W
Valeur par défaut	0	0	0	0	0	0	0	-



6) La file des 32 registres généraux

- La file de registres est optimisée pour supporter des opérations de type RISC :
  - Une opérande de sortie 8 bits et une entrée sur 8 bits
  - Deux opérandes de sortie 8 bits et une entrée sur 8 bits
  - Deux opérandes de sortie sur 8 bits et une entrée sur 16 bits
  - Une opérande de sortie sur 16 bits et une entrée sur 16 bits
- La plupart des opérations sur la file de registres ont un accès direct à ces registres et s'exécutent en un seul cycle

Les 32 registres de travail à usage général



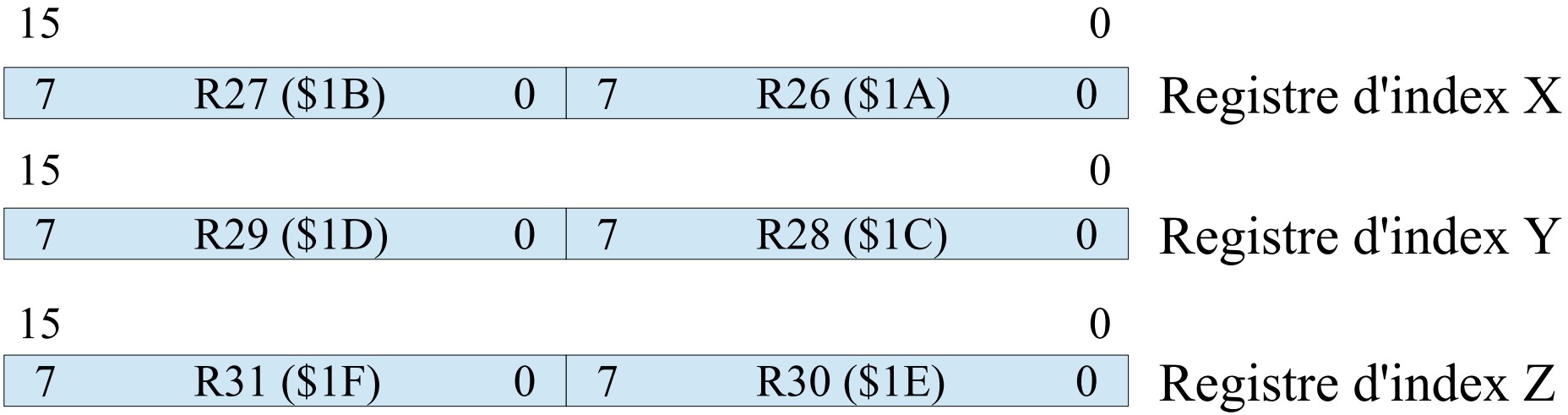
Adresses

0x00
0x01
0x0C
0x0D
0x0E
0x0F
0x10
0x11
0x1A
0x1B
0x1C
0x1D
0x1E
0x1F

ld / st

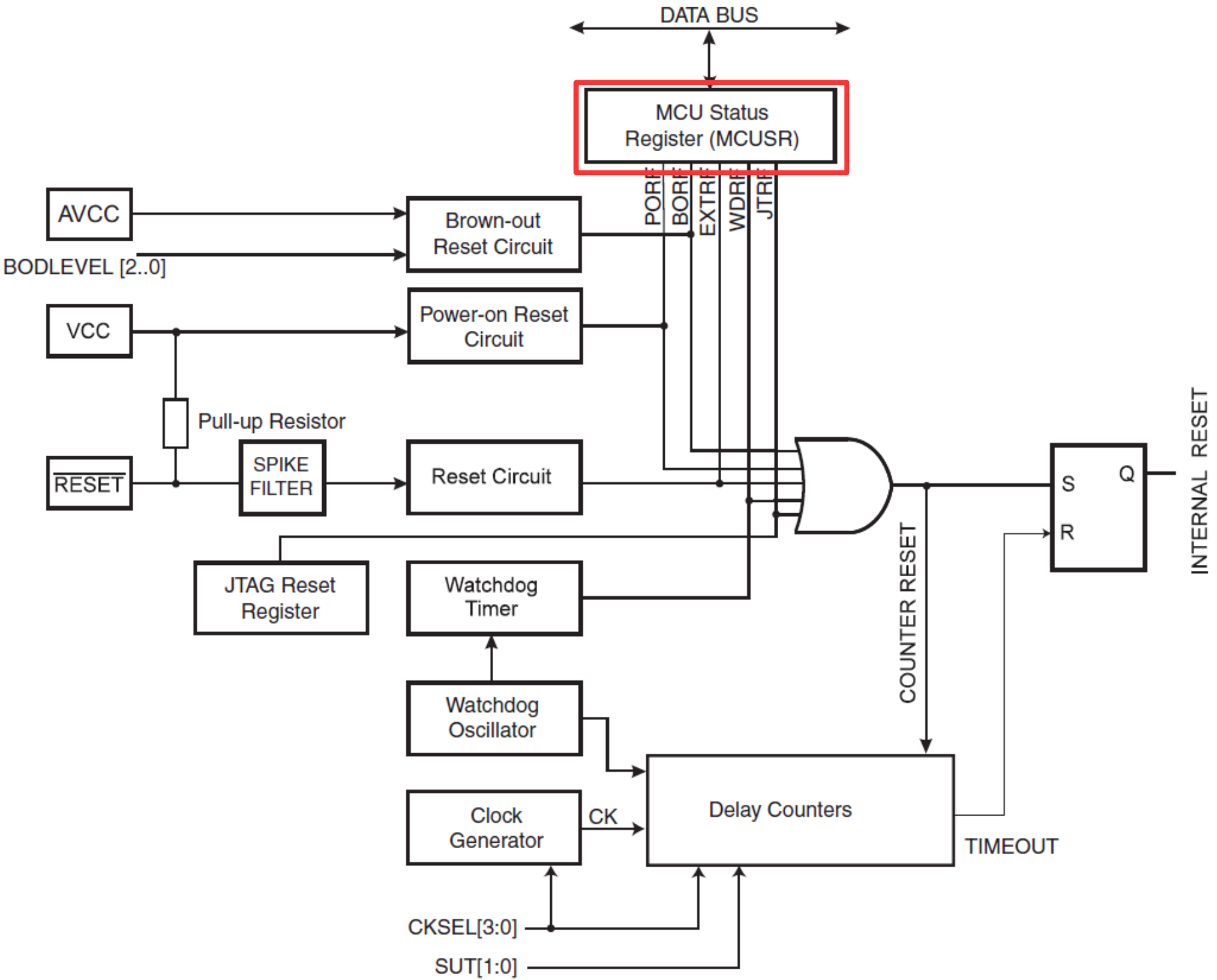
X – registre poids faible
X – registre poids fort
Y – registre poids faible
Y – registre poids fort
Z – registre poids faible
Z – registre poids fort

Les trois registres d'index (X, Y et Z) :



7) Le registre d'état spécial

- Le registre d'état spécial (MCUSR : MCU Status Register) permet d'identifier la source d'un Reset du processeur
- Il est constitué de 5 bits (les bits 0 à 4) qui sont :
  - PORF (bit 0) : pour indiquer la mise sous tension
  - EXTRF (bit 1) : pour l'entrée Reset externe
  - BORF (bit 2) : Reset provoqué par la circuitrie de reset
  - WDRF (bit 3) : pour indiquer un Watch Dog Reset
  - JTRF (bit 4) : pour un JTAG reset

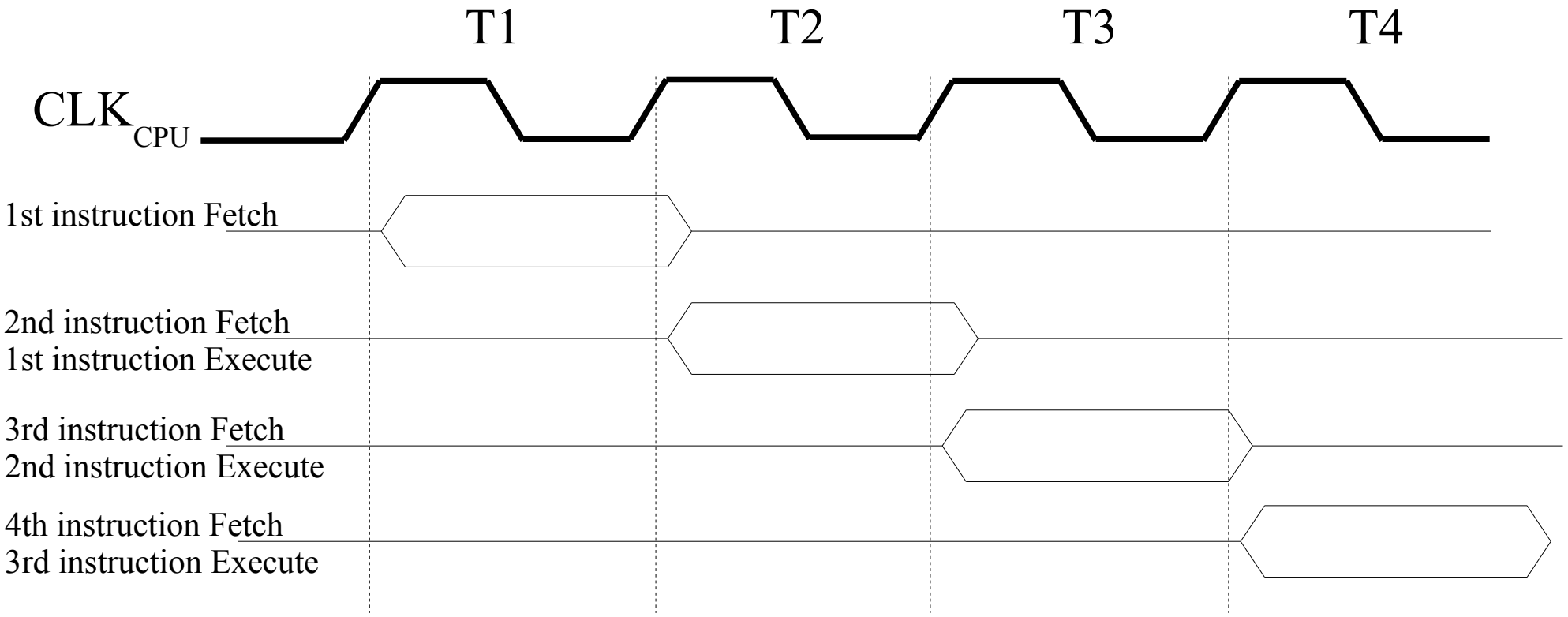

DATA BUS
MCU Status Register (MCUSR)
PORF
BORF
EXTRF
WDRF
JTRF
AVCC
BODLEVEL [2..0]
Brown-out Reset Circuit
VCC
Power-on Reset Circuit
Pull-up Resistor
RESET
SPIKE FILTER
Reset Circuit
JTAG Reset Register
Watchdog Timer
Watchdog Oscillator
Clock Generator
CK
Delay Counters
TIMEOUT
CKSEL[3:0]
SUT[1:0]
COUNTER RESET
S
Q
R
INTERNAL RESET

	Bits	7	6	5	4	3	2	1	0
MCUSR	0x54				JTRF	WDRF	BORF	EXTRF	PORF
	Read/Write	R	R	R	R/W	R/W	R/W	R/W	R/W
	Valeur par défaut	0	0	0	-	-	-	-	-

- Bit 7 – 5 : 0
- Bit 4 : JTRF
- Bit 3 : WDRF
- Bit 2 : BORF
- Bit 1 : EXTRF
- Bit 0 : PORF

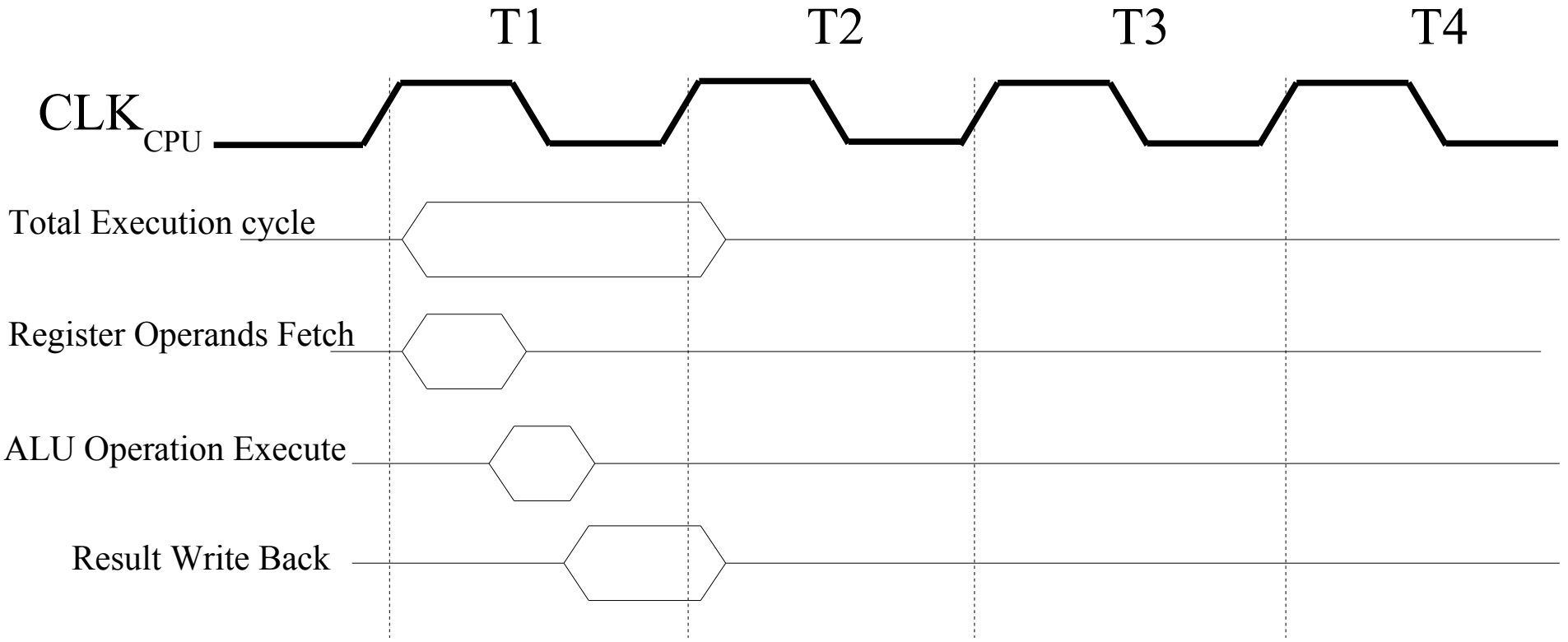
8) Les cycles du processeur AVR

- Le processeur AVR est cadencé par l'horloge du CPU $clock_{CPU}$ directement issue de l'horloge du circuit. Les cycles Fetch et Execute sont exécuté en parallèle.



8) Les cycles du processeur AVR

- La plupart des instructions s'exécutent en 1 cycle
- Cela rend le calcul des délai extrêmement simple
  - Pas de cache



4. Les 13 modes d'adressage

4. Les 13 modes d'adressage

- Le micro-contrôleur ATmega2560 permet de réaliser à peu près toutes les opérations de calcul, de manipulation de bits, d'accès aux entrées-sorties
- L'ensemble de ces opérations ne peut se faire qu'à l'aide de modes d'adressages qui vont du plus simple jusqu'au plus complexe

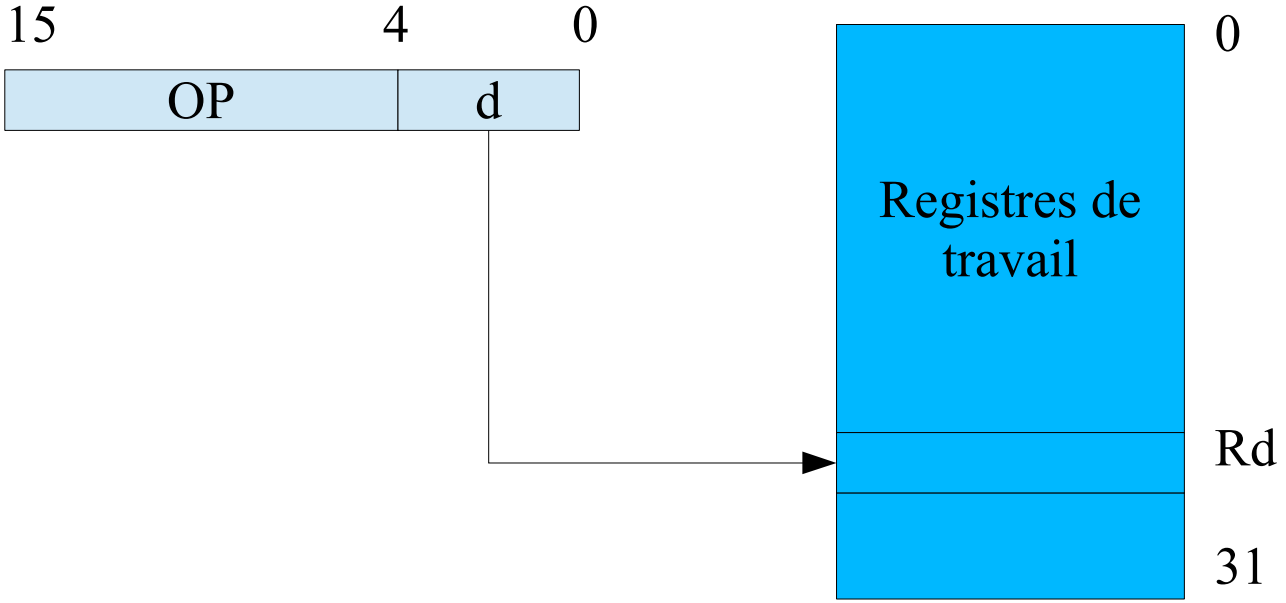
4. Les 13 modes d'adressage

- Le mot de 16 bits représenté dans ces schémas et d'où partent ou arrivent les données correspond au mot de code d'une instruction
- L'indication OP correspond au champ de l'instruction qui représente l'opération proprement dite
- Les références à d, r, p ou k sont des constantes spécifiées directement dans les instructions

4.1) Le mode d'adressage direct

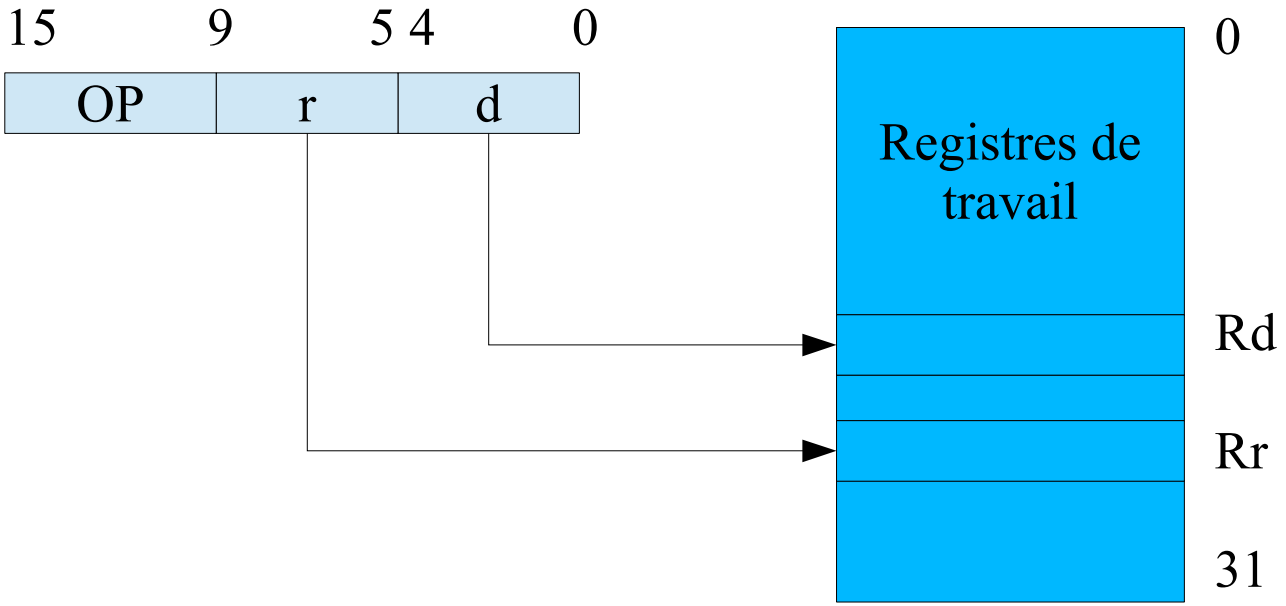
1.) Adressage direct simple registre

- C'est le plus simple des modes d'adressages, l'opérande manipulée est contenue dans un registre de travail Rd



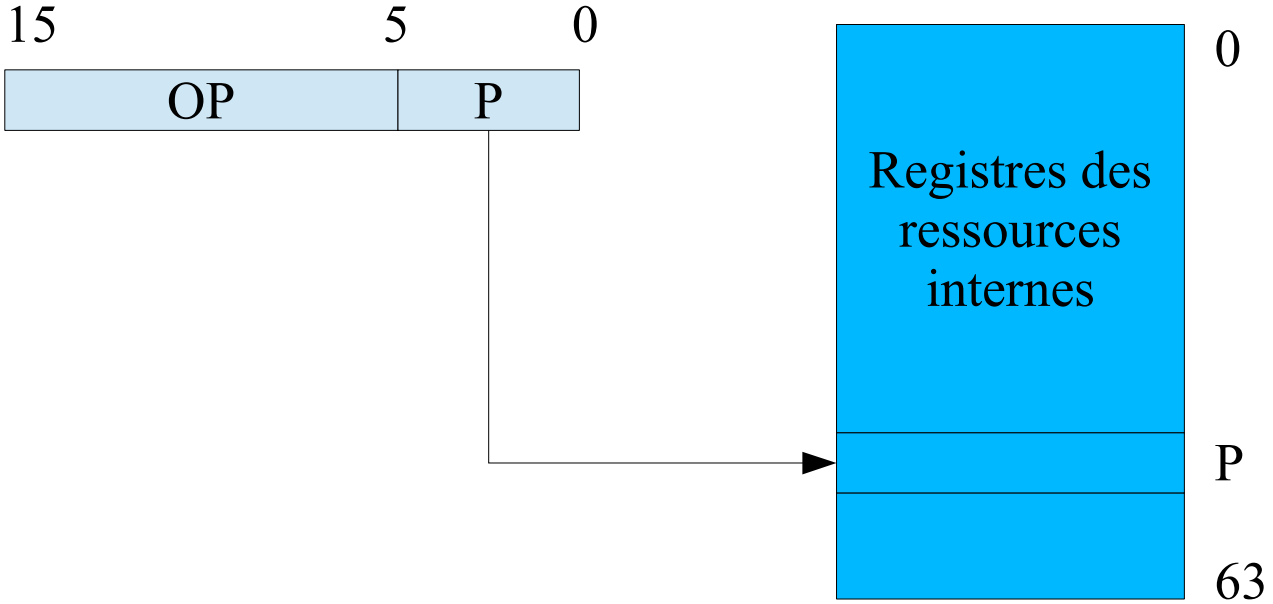
2.) Adressage direct double registre

- C'est une extension du mode d'adressage simple registre. Les opérandes manipulées sont contenues dans les registres de travail Rd et Rr.



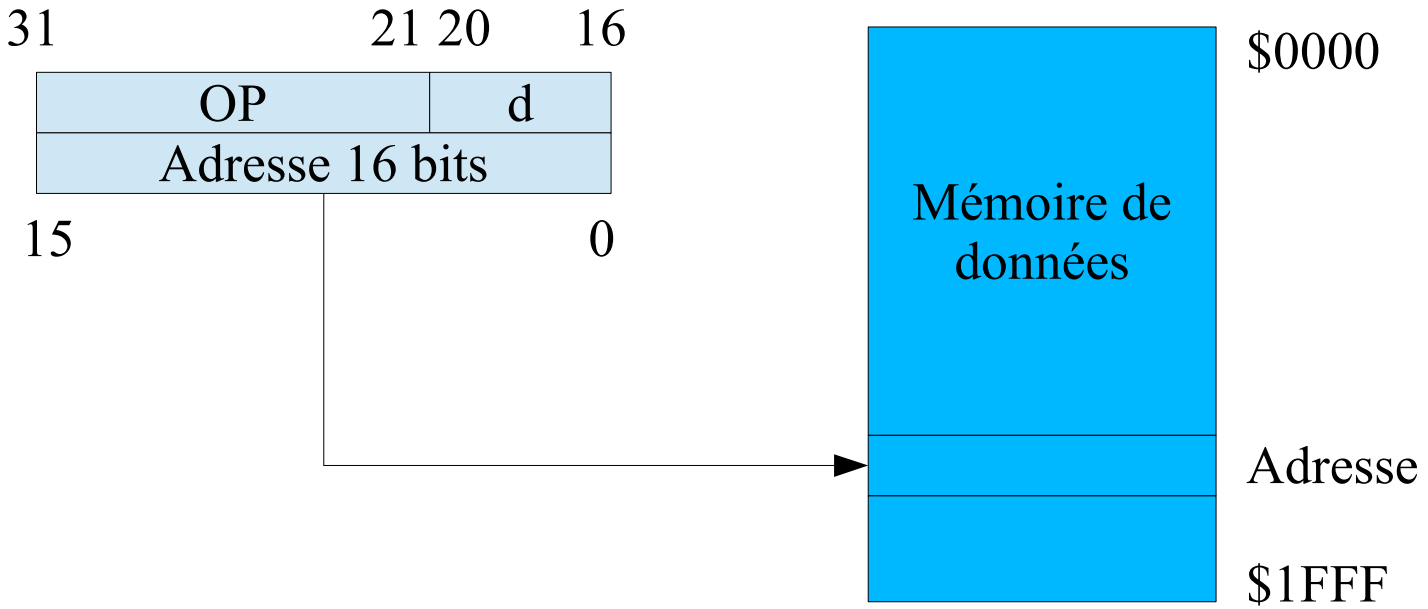
3.) Adressage direct des entrées-sorties

- Identique à l'adressage direct simple registre mais fait référence aux registres des ressources internes, au nombre de 64.



4.) Adressage direct de la mémoire de données

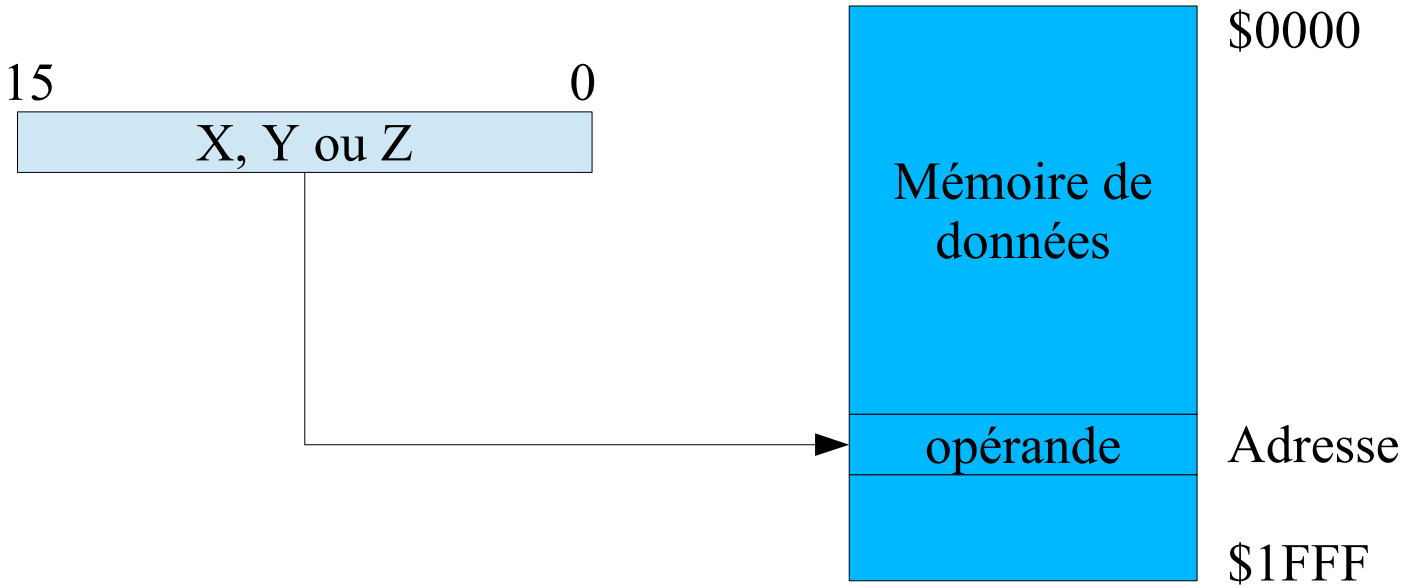
- L'instruction codée sur deux mots de 16 bits contient directement une adresse d'une opérande située en mémoire de données. Le premier mot peut également contenir un numéro de registre de données.



4.2) Mode d'adressage indirect de la mémoire de données

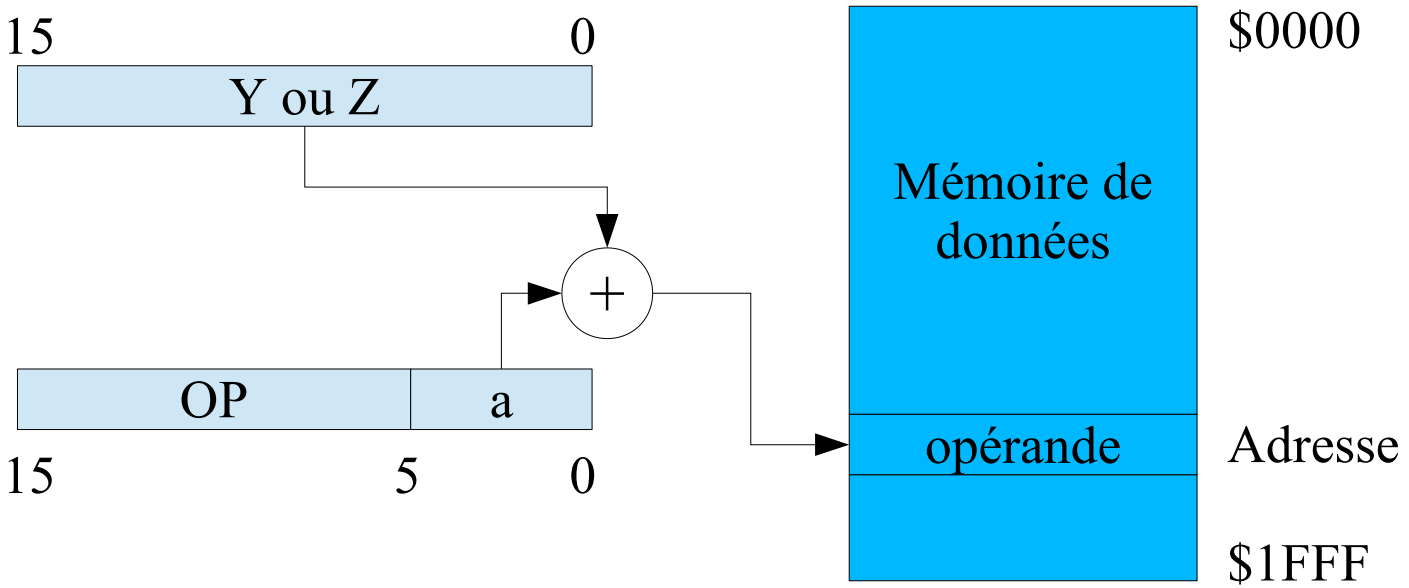
1.) Adressage indirect simple

- La position mémoire de l'opérande n'est plus située directement dans le mot qui suit l'instruction mais plutôt dans l'un des registres X, Y ou Z.



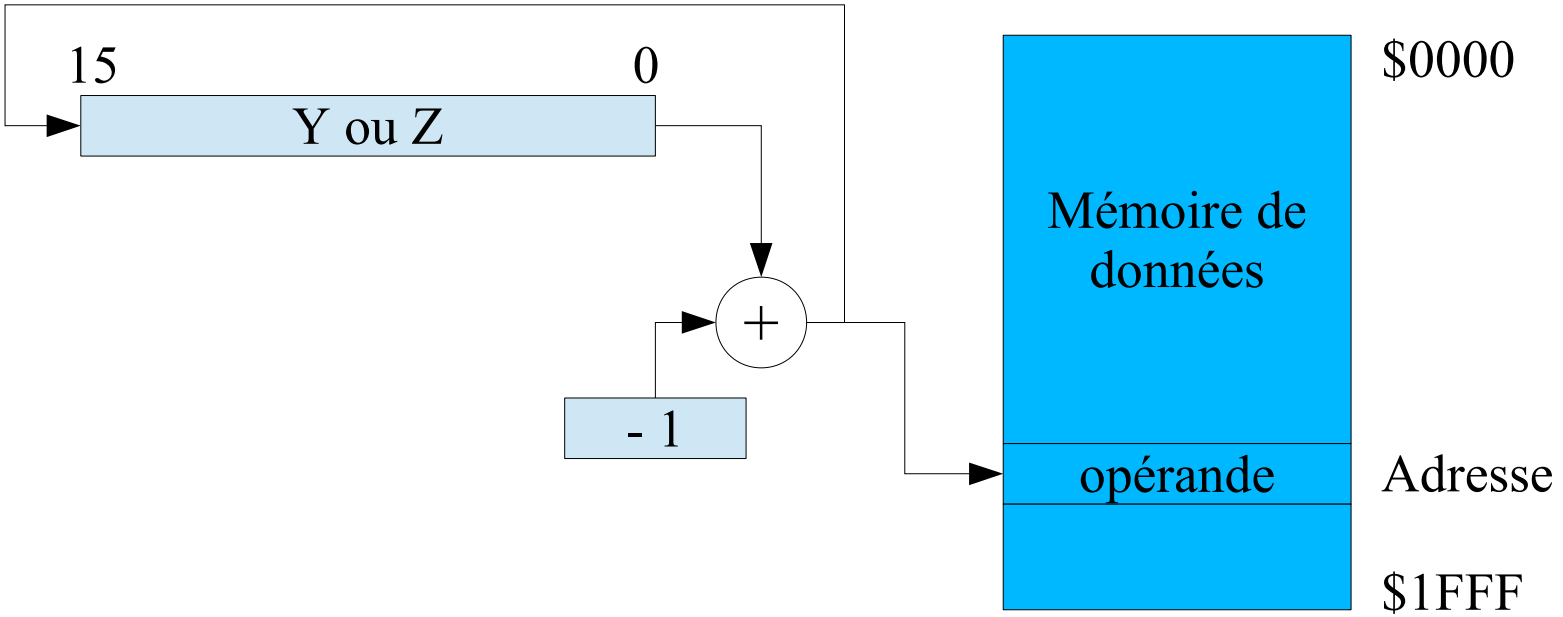
2.) Adressage indirect avec déplacement

- L'adresse de l'opérande est obtenue en ajoutant le contenu d'un des registres d'index Y ou Z à une constante a appelée déplacement. Cette constante est codée sur 6 bits.



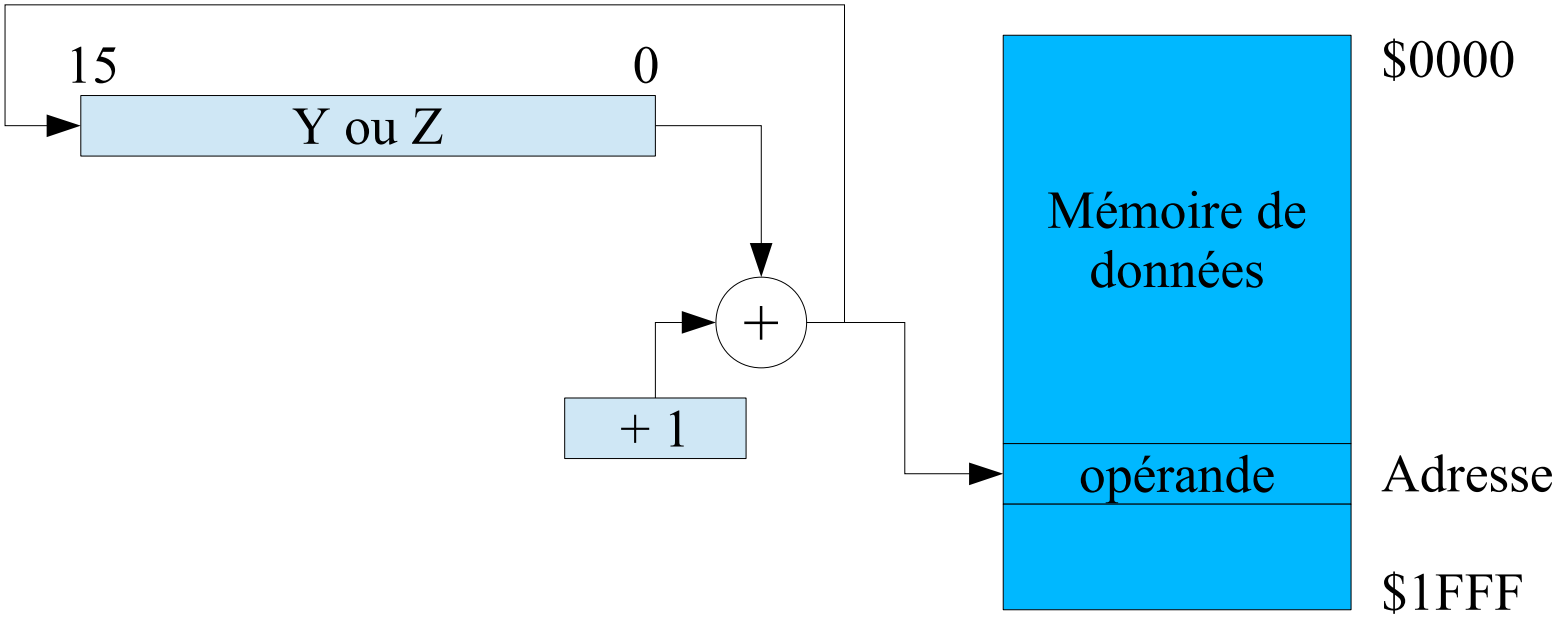
3.) Adressage indirect avec pré-décrémentation

- Ce mode d'adressage est une version évoluée du précédent très utile pour parcourir une table de valeurs. Le contenu du registre est diminué d'une unité avant l'exécution de l'instruction.



4.) Adressage indirect avec post-incrémentation

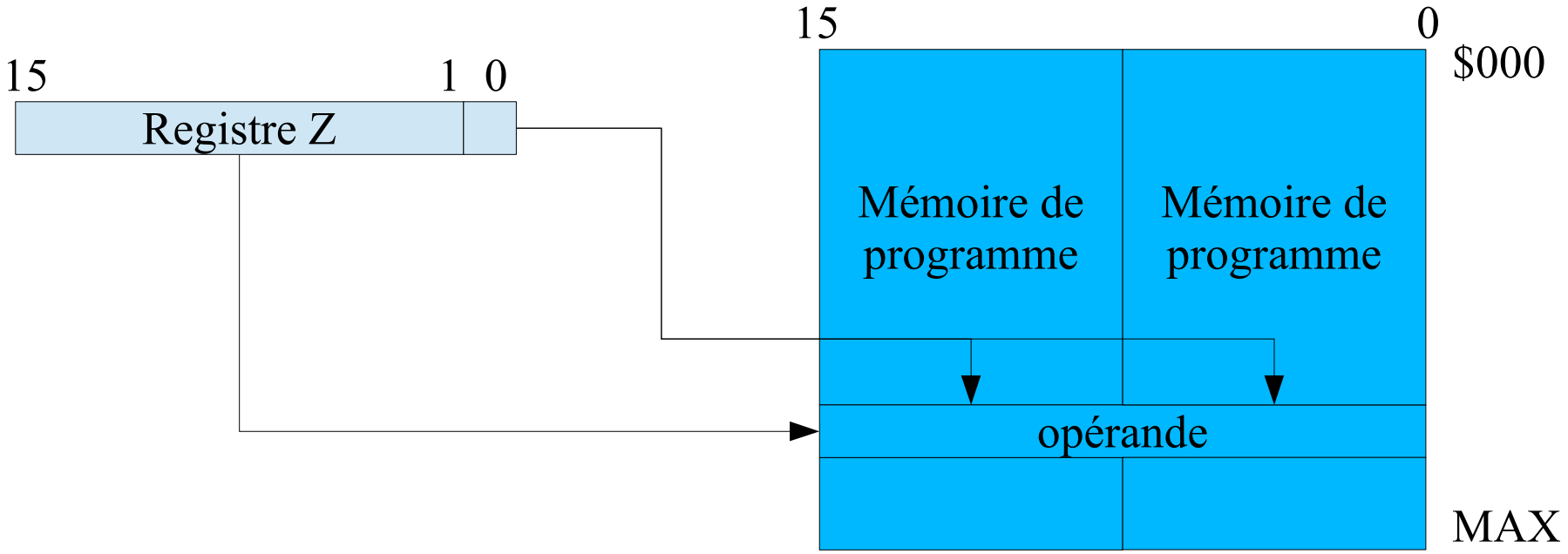
- Tout à fait similaire au mode indirect avec pré-décrémentation, celui-ci correspond à une adresse qui est le contenu du registre d'index, puis l'adresse est incrémenté d'une unité.



4.3) Mode d'adressage de la mémoire de programme

1.) Adressage constant utilisant les instructions LPM, ELPM et SPM

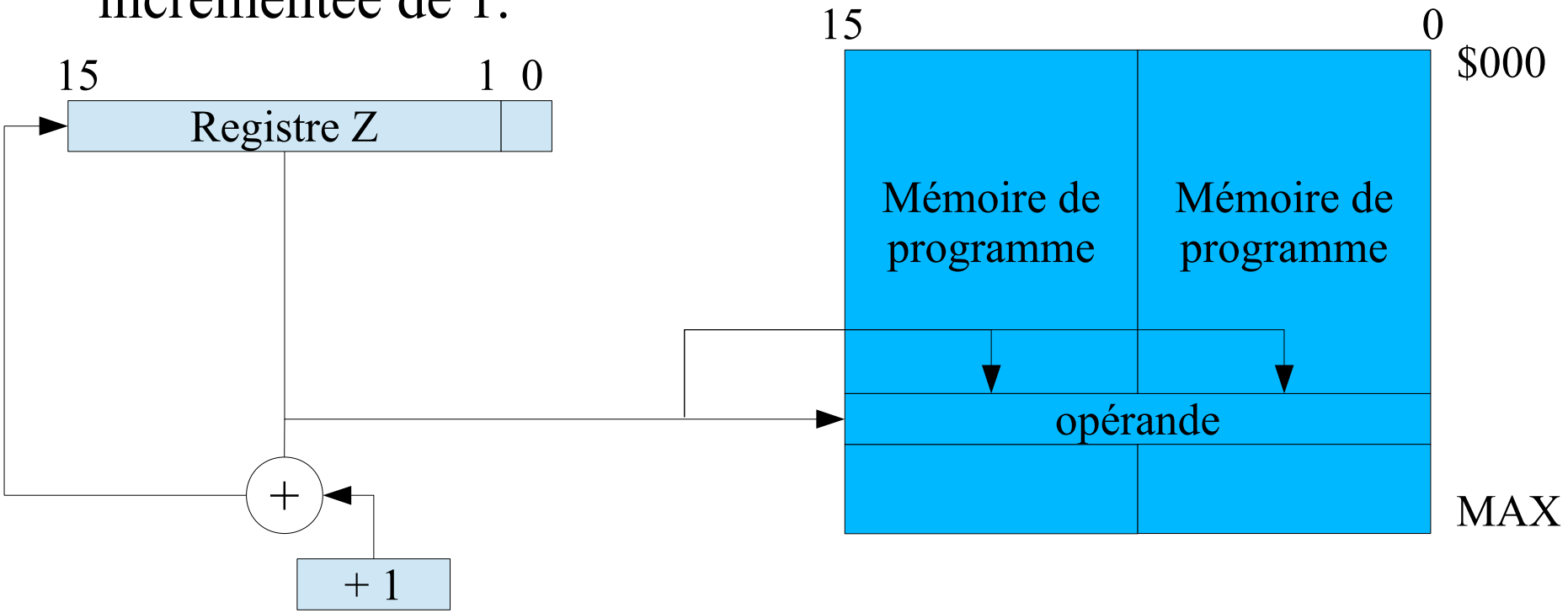
- Les 15 bits de poids fort du registre d'index Z contiennent une adresse d'opérande alors que le bit de poids faible* correspond à l'octet.



*Dans le cas d'une instruction LPM. Pour une instruction SPM ce bit est toujours à 0. Pour l'instruction ELPM ou SPM c'est le registre RAMPZ qui étend la taille du registre Z.

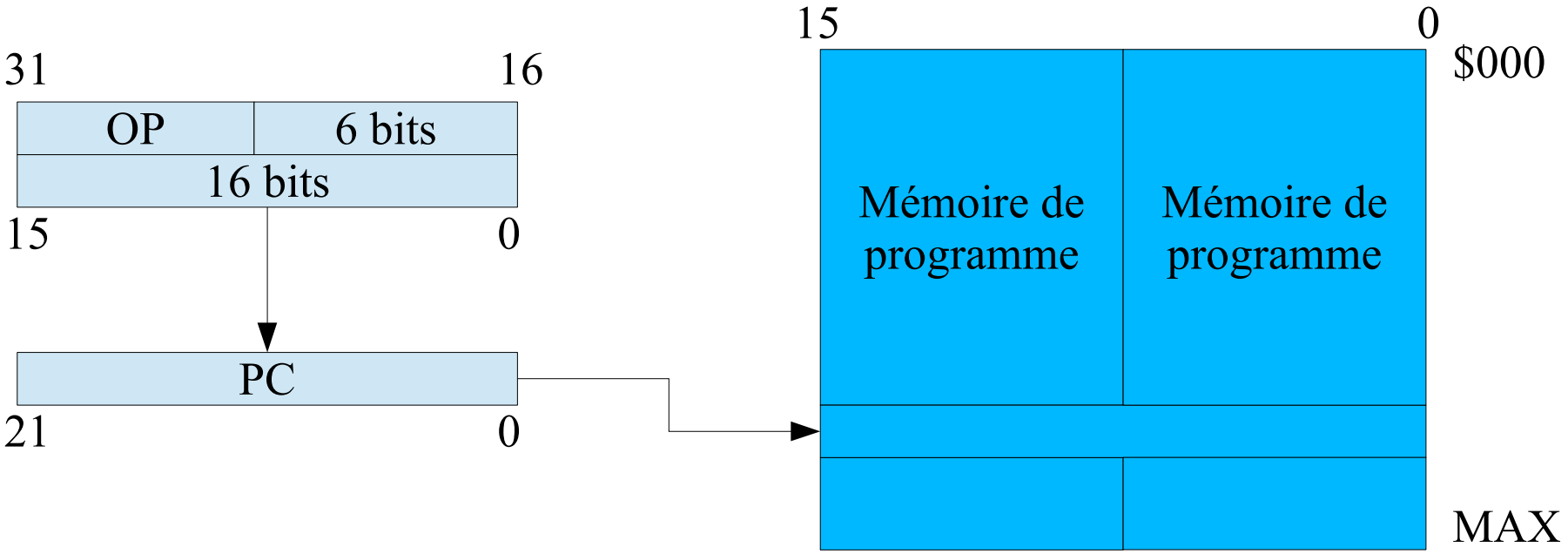
2.) Adressage post-incrémenté utilisant les instruction LPM et ELPM

- Les 15 bits de poids fort du registre d'index Z contiennent l'adresse d'une opérande alors que le bit de poids faible fait accéder à l'octet dans la mémoire de programme. Pour l'instruction ELPM c'est le bit du registre RAMPZ qui est utilisé. Une fois l'instruction exécutée, l'adresse est incrémentée de 1.



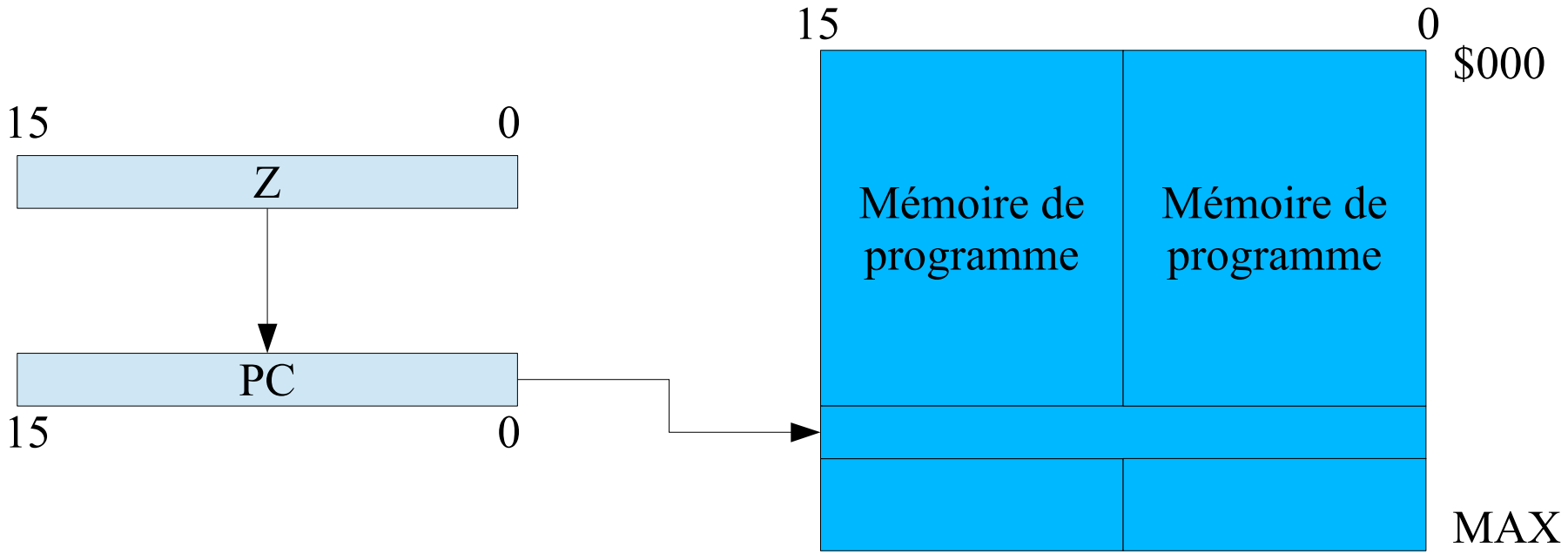
3.) Adressage direct de la mémoire de programme

- Ce mode d'adressage n'intervient que lors de l'exécution d'instruction JMP ou CALL. IL a pour effet de charger le compteur ordinal (PC) avec la valeur située immédiatement après l'instruction de saut.



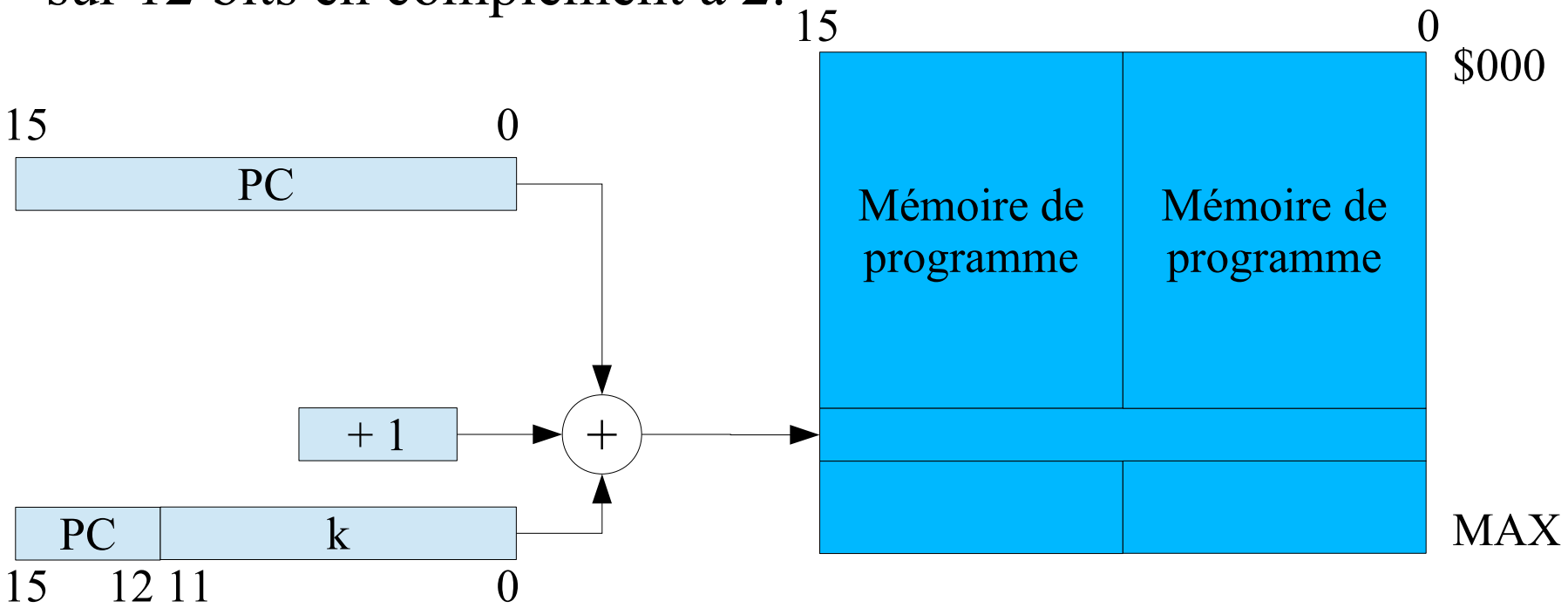
4.) Adressage indirect utilisant IJMP et ICALL

- Ce mode d'adressage n'intervient que lors de l'exécution d'instruction IJMP ou ICALL. IL a pour effet de charger le compteur ordinal (PC) avec la valeur du registre d'indirection Z.



5.) Adressage relatif de la mémoire de programme

- Ce mode d'adressage est celui le plus utilisé lors de l'emploi des instructions de saut ou de branchement. Le code de l'instruction contient une donnée, appelée k, qui vient s'ajouter à la valeur du PC original. Ce déplacement est codé sur 12 bits en complément à 2.



5. Le jeu d'instruction de l'AVR ATmega2560

1. registre DDRLx1

si $DDRX_n = 1$ => Sortie

$X_n = 0$ => Entrée

si sortie | $PORTx_n = 1$ => 1 correspond à 5v

$X_n = 0$ => 0

$X_n = X$ => (en écriture)

si entrée | $Pinx_n = X$ => (en lecture)

$X_n = 1$ => 1 pour 5v

$X_n = 0$ => 0

Les instructions arithmetiques de l'AVR

Mnémonique	Ops	Description	Opération	Flags	#Clocks
ADD	Rd, Rr	Addition	Rd<-Rd+Rr	H,S,V,N,Z,C	1
ADC	Rd, Rr	Addition with carry	Rd<-Rd+Rr+C	H,S,V,N,Z,C	1
ADIW	Rd, K	Addition	Rd:Rd+1<-Rd+1:Rd+K	S,V,N,Z,C	2
SUB	Rd, Rr	Substraction	Rd<-Rd - Rr	H,S,V,N,Z,C	1
SUBI	Rd, K	Substraction	Rd<-Rd - K	H,S,V,N,Z,C	2
SBC	Rd, Rr	Substraction w/ C	Rd<-Rd-Rr-C	H,S,V,N,Z,C	1
SBCI	Rd, K	Substraction	Rd<-Rd-K-C	H,S,V,N,Z,C	2
SBIW	Rd, K	Substraction	Rd:Rd+1<-Rd+1:Rd-K	S,V,N,Z,C	2
INC	Rd	Increment	Rd<-Rd+1	S,V,N,Z	1
DEC	Rd	Decrement	Rd<-Rd-1	S,V,N,Z	1
MUL	Rd, Rr	Multiply	R1:R0<-RdxRr	Z,C	2
MULS	Rd, Rr	Mult. Signed	R1:R0<-RdxRr	Z,C	2
MULSU	Rd, Rr	Mult. Unsign	R1:R0<-RdxRr	Z,C	2
FMUL	Rd, Rr	Frac mul	R1:R0<-RdxRr	Z,C	2
FMULS	Rd, Rr	Frac mul sign	R1:R0<-RdxRr	Z,C	2
FMULSU	Rd, Rr	Frac mul uns.	R1:R0<-RdxRr	Z,C	2

Les instructions logiques de l'AVR

Mnémonique	Ops	Description	Opération	Flags	#Clocks
AND	Rd, Rr	Logical AND	Rd ← Rd.Rr	S,V,N,C	1
ANDI	Rd, K	Logical AND	Rd ← Rd.K	S,V,N,C	1
OR	Rd, Rr	Logical OR	Rd ← Rd+Rr	S,V,N,Z	1
ORI	Rd, K	Logical OR	Rd ← Rd+K	S,V,N,Z	1
EOR	Rd, Rr	Exclusive OR	Rd ← Rd+Rr	S,V,N,Z	1
COM	Rd	Complement	Rd ← $FF-Rd	S,V,N,Z	1
NEG	Rd	Negate	Rd ← 0-Rd	H,S,V,N,Z,C	1
SBR	Rd, K	Set bits	Rd ← Rd ou K	S,V,N,Z	1
CBR	Rd, K	Clear bits	Rd ← Rd et ($FF-K)	S,V,N,Z	1
TST	Rd	Test bits	Rd et Rd	S,V,N,Z	1
CLR	Rd	Clear bits	Rd ← 0	S,V,N,Z	1
SER	Rd	Set all bits	Rd ← $FF		1

Les instructions de saut de l'AVR

Mnémonique	Ops	Description	Opération	Flags	#Clocks
RJMP	K	Rel Jump	PC ← PC+K+1		2
IJMP		Ind jump	PC ← Z		2
EIJMP		Ext ind jump	PC ← EIND:Z		2
JMP	K	Jump	PC ← K		3
RCALL	K	Rel. Call	PC ← PC+K+1		4
ICALL		Ind call	PC ← Z		4
EICALL		Ext. Ind call	PC ← EIND:Z		4
CALL	K	Long call	PC ← K		5
RET		Ret. subr.	PC ← pop		5
RETI		Ret. Inter.	PC ← pop	I	5

Les instructions de skip et branch de l'AVR

Mnem.	Ops	Description	Opération	Flags	#Clocks
CPSE	Rd, Rr	Comp. and skip	Si Rd=Rr PC ← PC+2/3		1/2/3
CP	Rd, Rr	Compare	Rd - Rr	H,S,V,N,Z,C	1
CPC	Rd, Rr	Comp. w/ C	Rd - Rr - C	H,S,V,N,Z,C	1
CPI	Rd, K	Compare	Rd - K	H,S,V,N,Z,C	1
SBRC	Rd, b	Skip if not bit	Si Rd(b)=0 PC ← PC+2/3		1/2/3
SBRS	Rd, b	Skip if bit	Si Rd(b)=1 PC ← PC+2/3		1/2/3
SBIC	A, b	Skip if not I/O bit	Si IO(A,b)=0 PC ← PC+2/3		1/2/3
SBIS	A, b	Skip if I/O bit	Si IO(A,b)=1 PC ← PC+2/3		1/2/3
BRBS	s, K	Br. if Bit=1	Si SREG(s)=1 PC ← PC+K+1		1/2
BRBC	s, K	Br if Bit=0	Si SREG(s)=0 PC ← PC+K+1		1/2
BREQ	K	Br. if Equal	Si Z=1 PC ← PC+K+1		1/2
BRNE	K	Br. if not equal	Si Z=0 PC ← PC+K+1		1/2
BRCS	K	Br. if carry set	Si C=1 PC ← PC+K+1		1/2
BRCC	K	Br. if carry clear	Si C=0 PC ← PC+K+1		1/2
BRSH	K	Br. if half carry	Si H=1 PC ← PC+K+1		1/2
BRLO	K	Br. if lower	Si C=0 PC ← PC+K+1		1/2

Les instructions de skip et branch de l'AVR

Mném.	Ops	Description	Opération	Flags	#Clocks
BRMI	K	Br. if minus	Si N=1 PC ← PC+K+1		1/2
BRPL	K	Br. if plus	Si N=0 PC ← PC+K+1		1/2
BRGE	K	Br. if >=0	Si S=0 PC ← PC+K+1		1/2
BRLT	K	Br. if <0	Si S=1 PC ← PC+K+1		1/2
BRHS	K	Br. if half carry	Si H=1 PC ← PC+K+1		1/2
BRHC	K	Br. if not half carry	Si H=0 PC ← PC+K+1		1/2
BRTS	K	Br. if T=1	Si T=1 PC ← PC+K+1		1/2
BRTC	K	Br. if T=0	Si T=0 PC ← PC+K+1		1/2
BRVS	K	Br. if V=1	Si V=1 PC ← PC+K+1		1/2
BRVC	K	Br. if V=0	Si V=0 PC ← PC+K+1		1/2
BRIE	K	Br. if I=1	Si I=1 PC ← PC+K+1		1/2
BRID	K	Br. if I=0	Si I=0 PC ← PC+K+1		1/2

Les instructions de shift et bit de l'AVR

Mném.	Ops	Description	Opération	Flags	#Clocks
LSL	Rd	Logical Shift Left	Rd(n+1) ← Rd(n)	Z,C,N,V	1
LSR	Rd	Logical Shift Right	Rd(n) ← Rd(n+1)	Z,C,N,V	1
ROL	Rd	Rotate Left w/ C		Z,C,N,V	1
ROR	Rd	Rotate Right w/ C		Z,C,N,V	1
ASR	Rd	Arithmetic Shift Right	Rd(n) ← Rd(n+1)	Z,C,N,V	1
SWAP	Rd	Swap Nibbles			1
BSET	s	Flag Set	Sreg(s) ← 1	SREG(s)	1
BCLR	s	Flag Clear	Sreg(s) ← 0	SREG(s)	1
SBI	P, b	Set Bit in I/O Reg.	I/O(P,b) ← 1		1
CBI	P, b	Clear Bit in I/O Reg.	I/O(P,b) ← 0		1
BST	Rd, b	Bit Store Reg. In T	T ← Rd(b)	T	1
BLD	Rd, b	Bit Load T to Reg.	Rd(b) ← T		1
SEC		Set Carry	C ← 1	C	1
CLC		Clear Carry	C ← 0	C	1
SEN		Set Negative Flag	N ← 1	N	1
CLN		Clear Negative Flag	N ← 0	N	1

Les instructions de shift et bit de l'AVR

Mném.	Ops	Description	Opération	Flags	#Clocks
SEZ		Set Zero Flag	Z ← 1	Z	1
CLZ		Clear Zero Flag	Z ← 0	Z	1
SEI		Set Inter. Flag	I ← 1	I	1
CLI		Clear Inter. Flag	I ← 0	I	1
SES		Set Sign. Flag	S ← 1	S	1
CLS		Clear Sign. Flag	S ← 0	S	1
SEV		Set V Flag	V ← 1	V	1
CLV		Clear V Flag	V ← 0	V	1
SET		Set T in Sreg	T ← 1	T	1
CLT		Clear T in Sreg	T ← 0	T	1
SEH		Set H Flag	H ← 1	H	1
CLH		Clear H Flag	H ← 0	H	1

Les instructions de chargement de l'AVR

Mném.	Ops	Description	Opération	#Clocks
LDI	Rd, K	Load imm.	Rd ← K	1
LDS	Rd, K	Load Direct from SRAM	Rd ← (K)	2
LD	Rd, X	Load indirect	Rd ← (X)	2
LD	Rd, X+	Load indirect w/ Post-increment	Rd ← (X), X ← X+1	2
LD	Rd, -X	Load indirect w/ Pre-decrement	X ← X-1, Rd ← (X)	2
LD	Rd, Y	Load indirect	Rd ← (Y)	2
LD	Rd, Y+	Load indirect w/ Post-increment	Rd ← (Y), Y ← Y+1	2
LD	Rd, -Y	Load indirect w/ Pre-decrement	Y ← Y-1, Rd ← (Y)	2
LDD	Rd, Y+q	Load indirect with Displacement	Rd ← (Y+q)	2
LD	Rd, Z	Load indirect	Rd ← (Z)	2
LD	Rd, Z+	Load indirect w/ Post-increment	Rd ← (Z), Z ← Z+1	2
LD	Rd, -Z	Load indirect w/ Pre-decrement	Z ← Z-1, Rd ← (Z)	2
LDD	Rd, Z+q	Load indirect w/ Displacement	Rd ← (Z+q)	2

Les instructions de stockage de l'AVR

Mnémonique	Ops	Description	Opération	#Clocks
STS	k, Rr	Store Direct to SRAM	(k) ← Rr	2
ST	X, Rr	Store indirect	(X) ← Rr	2
ST	X+, Rr	Store indirect w/ Post-increment	(X) ← Rr, X ← X+1	2
ST	-X, Rr	Store indirect w/ Pre-decrement	X ← X-1, (X) ← Rr	2
ST	Y, Rr	Store indirect	(Y) ← Rr	2
ST	Y+, Rr	Store indirect w/ Post-increment	(Y) ← Rr, Y ← Y+1	2
ST	-Y ,Rr	Store indirect w/ Pre-decrement	Y ← Y-1, (Y) ← Rr	2
STD	Y+q, Rr	Store indirect w/ Displacement	(Y+q) ← Rr	2
ST	Z, Rr	Store indirect	(Z) ← Rr	2
ST	Z+, Rr	Store indirect w/ Post-increment	(Z) ← Rr, Z ← Z+1	2
ST	-Z, Rr	Store indirect w/ Pre-decrement	Z ← Z-1, (Z) ← Rr	2
STD	Z+q, Rr	Store indirect w/ Displacement	(Z+q) ← Rr	2

Les instructions de load/store de l'AVR (mémoire de programme)

Mném.	Ops	Description	Opération	#Clocks
LPM		Load Program indirect Memory	R0 ← (Z)	3
LPM	Rd, Z	Load Program indirect Memory	Rd ← (Z)	3
LPM	Rd, Z+	Load Program Mem. w/ Post-inc	Rd ← (Z), Z ← Z+1	3
ELPM		Ext. Load Program ind. Mem	R0 ← (RAMPZ:Z)	3
ELPM	Rd, Z	Ext. Load Program ind. Mem	Rd ← (RAMPZ:Z)	3
ELPM	Rd, Z+	Ext. Load Prg. Mem. w/ Post-inc.	Rd ← (RAMPZ:Z), RAMPZ:Z ← RAMPZ:Z+1	3
SPM		Store Program indirect Memory	(Z) ← R1:R0	-

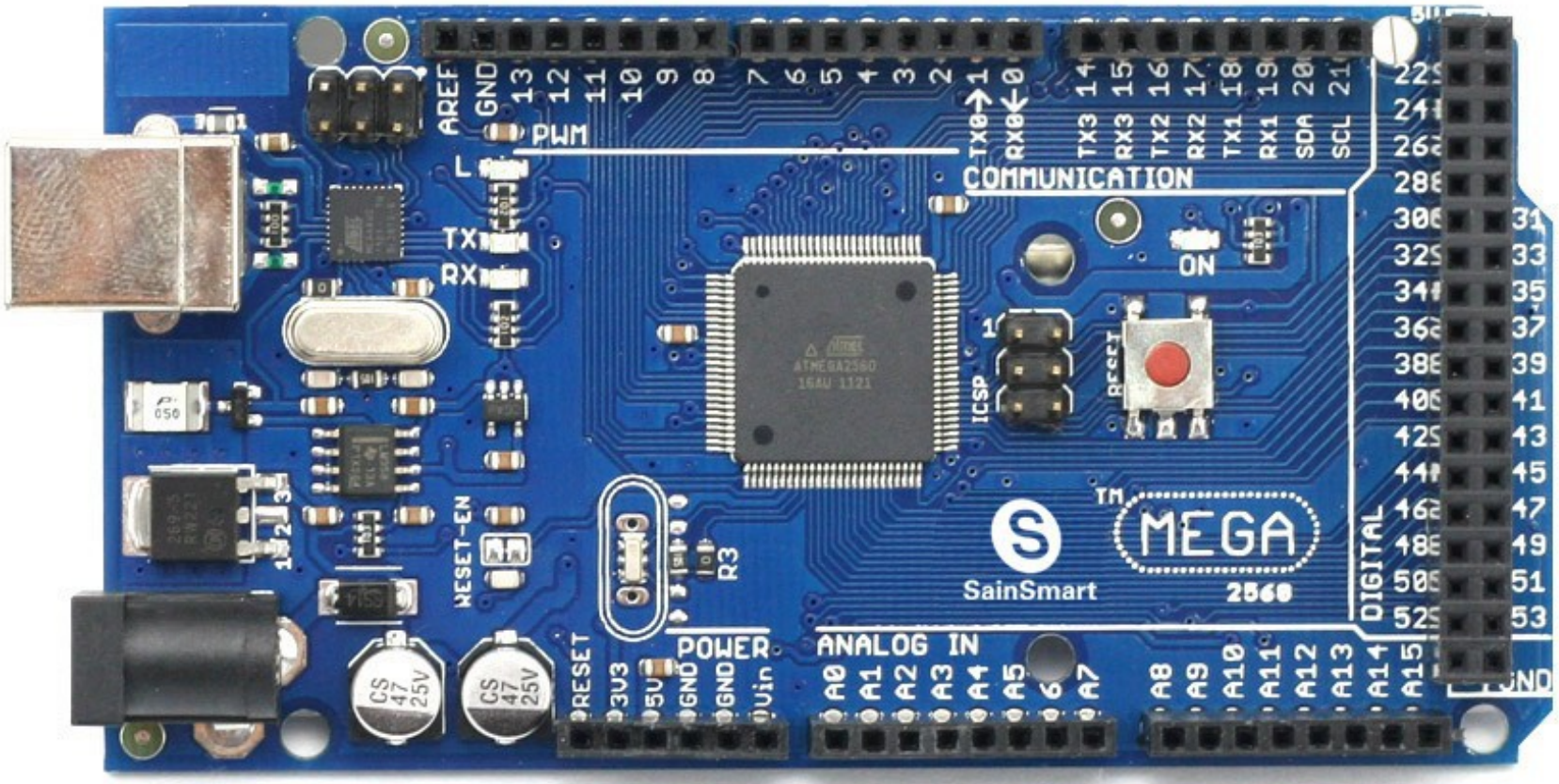
Les instructions de move et IO de l'AVR

Mném.	Ops	Description	Opération	#Clocks
MOV	Rd, Rr	Move betw. Reg.	Rd ← Rr	1
MOVW	Rd, Rr	Copy betw. Reg.	Rd+1:Rd ← Rr+1:Rr	1
IN	Rd, P	In Port	Rd ← P	1
OUT	P, Rr	Out Port	P ← Rr	1
PUSH	Rr	Push Register in Stack	STACK ← Rr	2
POP	Rd	Pop Register from Stack	Rd ← STACK	2

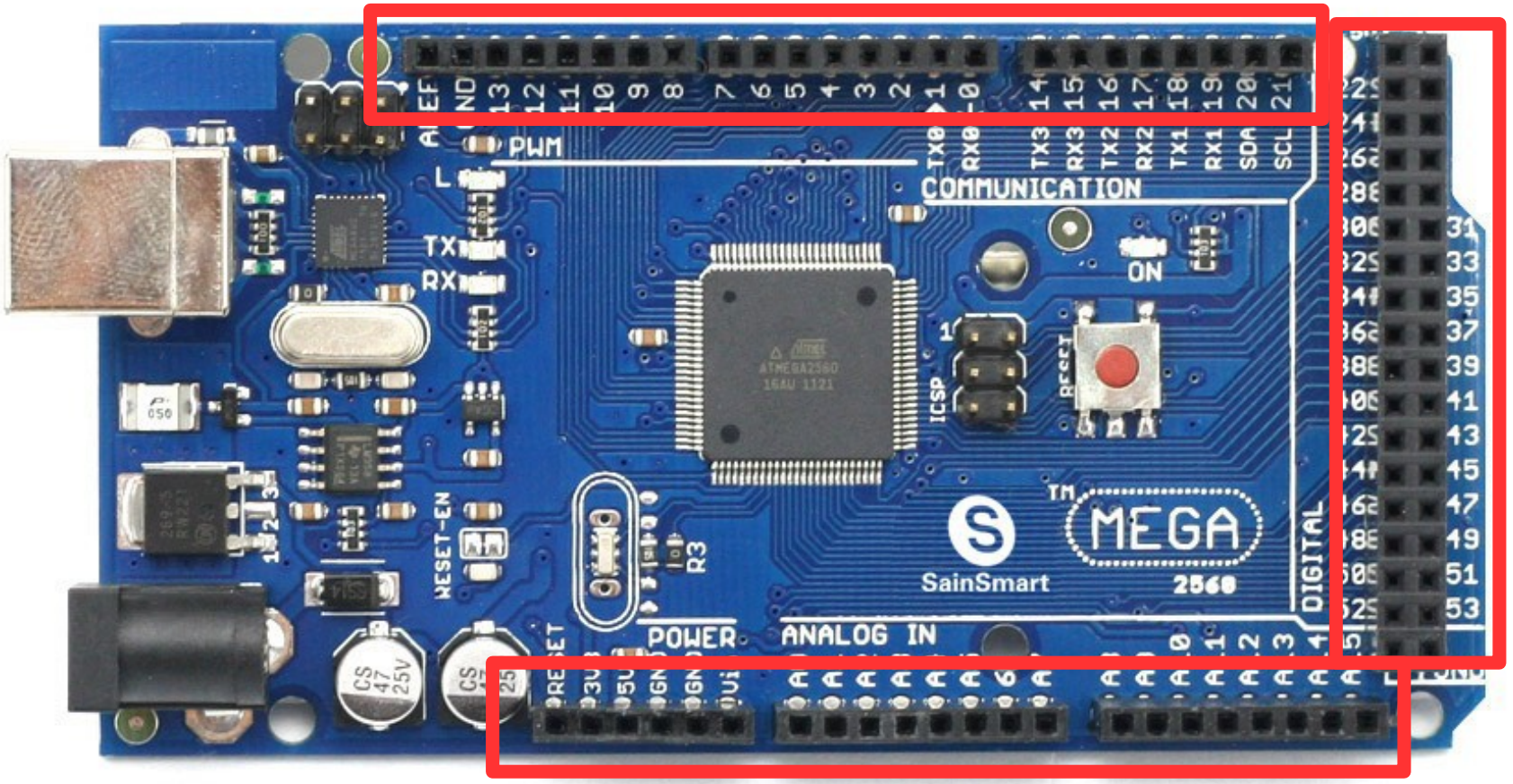
Les instructions de contrôle de l'AVR

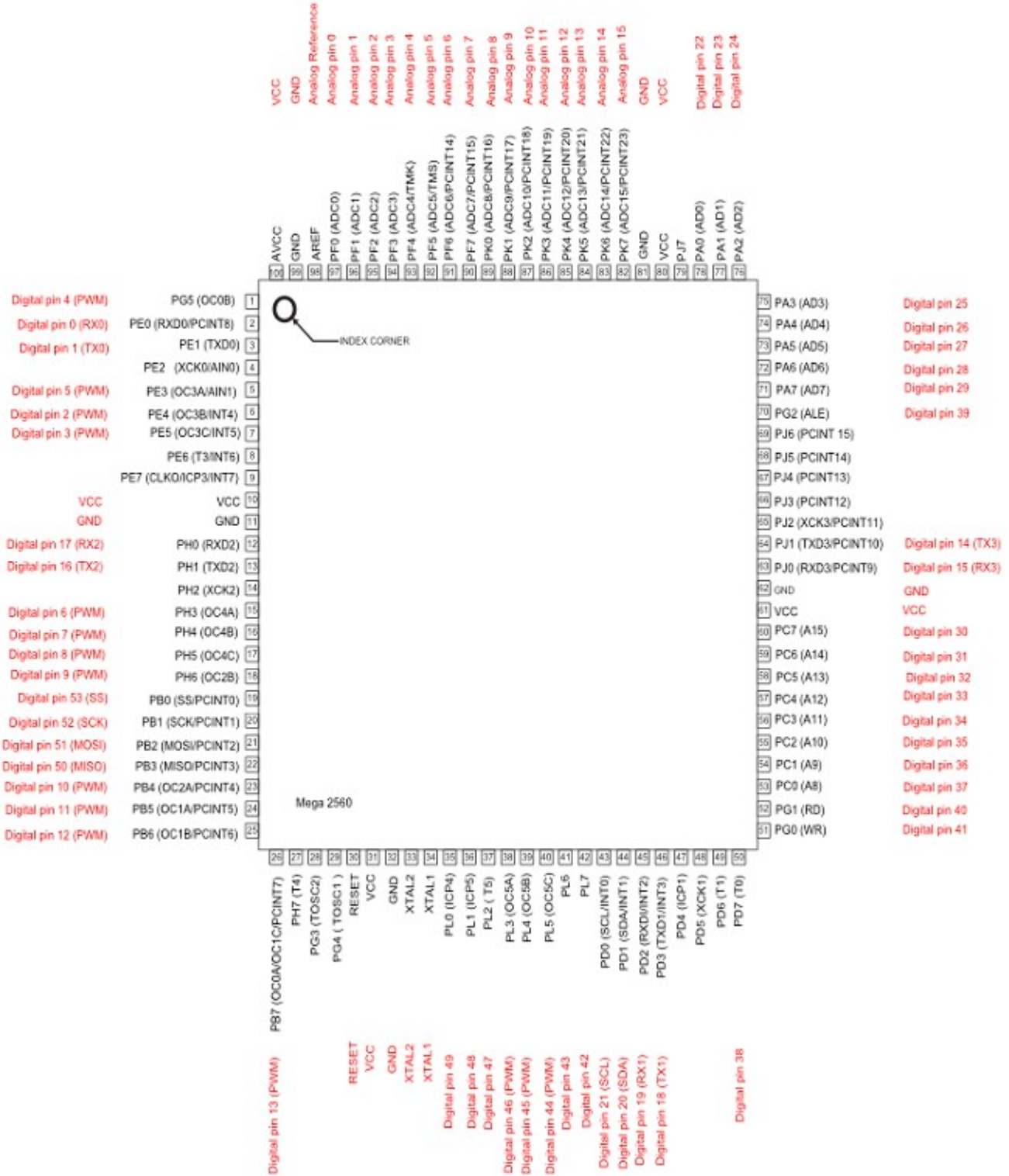
Mnémonique	Ops	Description	Opération	#Clocks
BREAK		Break	For OnChip Debug Only	N/A
NOP		No Operation		1
SLEEP		Sleep	See Sleep function	1
WDR		WatchDog Reset	See WDR/Timer function	1

6. Le brochage de la carte Arduino AVR ATmega2560



5. Le brochage de la carte Arduino AVR ATmega2560




Mega 2560
INDEX CORNER
Digital pin 4 (PWM) PG5 (OC0B) 1
Digital pin 0 (RX0) PE0 (RXD0/PCINT8) 2
Digital pin 1 (TX0) PE1 (TXD0) 3
PE2 (XCK0/AIN0) 4
Digital pin 5 (PWM) PE3 (OC3A/AIN1) 5
Digital pin 2 (PWM) PE4 (OC3B/INT4) 6
Digital pin 3 (PWM) PE5 (OC3C/INT5) 7
PE6 (T3/INT6) 8
PE7 (CLKO/ICP3/INT7) 9
VCC VCC 10
GND GND 11
Digital pin 17 (RX2) PH0 (RXD2) 12
Digital pin 16 (TX2) PH1 (TXD2) 13
PH2 (XCK2) 14
Digital pin 6 (PWM) PH3 (OC4A) 15
Digital pin 7 (PWM) PH4 (OC4B) 16
Digital pin 8 (PWM) PH5 (OC4C) 17
Digital pin 9 (PWM) PH6 (OC2B) 18
Digital pin 53 (SS) PB0 (SS/PCINT0) 19
Digital pin 52 (SCK) PB1 (SCK/PCINT1) 20
Digital pin 51 (MOSI) PB2 (MOSI/PCINT2) 21
Digital pin 50 (MISO) PB3 (MISO/PCINT3) 22
Digital pin 10 (PWM) PB4 (OC2A/PCINT4) 23
Digital pin 11 (PWM) PB5 (OC1A/PCINT5) 24
Digital pin 12 (PWM) PB6 (OC1B/PCINT6) 25
PB7 (OC0A/OC1C/PCINT7) 26
PH7 (T4) 27
PG3 (TOSC2) 28
PG4 (TOSC1) 29
RESET 30
VCC 31
GND 32
XTAL2 33
XTAL1 34
PL0 (ICP4) 35
PL1 (ICP5) 36
PL2 (T5) 37
PL3 (OC5A) 38
PL4 (OC5B) 39
PL5 (OC5C) 40
PL6 41
PL7 42
PD0 (SCL/INT0) 43
PD1 (SDA/INT1) 44
PD2 (RXD0/INT2) 45
PD3 (TXD1/INT3) 46
PD4 (ICP1) 47
PD5 (XCK1) 48
PD6 (T1) 49
PD7 (T0) 50
Digital pin 13 (PWM)
RESET
VCC
GND
XTAL2
XTAL1
Digital pin 49
Digital pin 48
Digital pin 47
Digital pin 46 (PWM)
Digital pin 45 (PWM)
Digital pin 44 (PWM)
Digital pin 43
Digital pin 42
Digital pin 21 (SCL)
Digital pin 20 (SDA)
Digital pin 19 (RX1)
Digital pin 18 (TX1)
Digital pin 38
AVCC 20
GND 99
AREF 98
PF0 (ADC0) 97
PF1 (ADC1) 96
PF2 (ADC2) 95
PF3 (ADC3) 94
PF4 (ADC4/TMK) 93
PF5 (ADC5/TMS) 92
PF6 (ADC6/PCINT14) 91
PF7 (ADC7/PCINT15) 90
PK0 (ADC8/PCINT16) 89
PK1 (ADC9/PCINT17) 88
PK2 (ADC10/PCINT18) 87
PK3 (ADC11/PCINT19) 86
PK4 (ADC12/PCINT20) 85
PK5 (ADC13/PCINT21) 84
PK6 (ADC14/PCINT22) 83
PK7 (ADC15/PCINT23) 82
GND 81
VCC 80
PJ7 79
PA0 (AD0) 78
PA1 (AD1) 77
PA2 (AD2) 76
VCC
GND
Analog Reference
Analog pin 0
Analog pin 1
Analog pin 2
Analog pin 3
Analog pin 4
Analog pin 5
Analog pin 6
Analog pin 7
Analog pin 8
Analog pin 9
Analog pin 10
Analog pin 11
Analog pin 12
Analog pin 13
Analog pin 14
Analog pin 15
GND
VCC
Digital pin 22
Digital pin 23
Digital pin 24
PA3 (AD3) 75
PA4 (AD4) 74
PA5 (AD5) 73
PA6 (AD6) 72
PA7 (AD7) 71
PG2 (ALE) 70
PJ6 (PCINT 15) 69
PJ5 (PCINT14) 68
PJ4 (PCINT13) 67
PJ3 (PCINT12) 66
PJ2 (XCK3/PCINT11) 65
PJ1 (TXD3/PCINT10) 64
PJ0 (RXD3/PCINT9) 63
GND 62
VCC 61
PC7 (A15) 60
PC6 (A14) 59
PC5 (A13) 58
PC4 (A12) 57
PC3 (A11) 56
PC2 (A10) 55
PC1 (A9) 54
PC0 (A8) 53
PG1 (RD) 52
PG0 (WR) 51
Digital pin 25
Digital pin 26
Digital pin 27
Digital pin 28
Digital pin 29
Digital pin 39
Digital pin 14 (TX3)
Digital pin 15 (RX3)
GND
VCC
Digital pin 30
Digital pin 31
Digital pin 32
Digital pin 33
Digital pin 34
Digital pin 35
Digital pin 36
Digital pin 37
Digital pin 40
Digital pin 41

Pin number	Pin Name	Map Pin Name	Pin on TinkerKit
1	PG5 (OC0B)	Digital pin 4 (PWM)	D4
2	PE0 (RXD0/PCINT8)	Digital pin 0 (PWM)	SERIAL0 (RXD)
3	PE1 (TXD0)	Digital pin 1 (TX0)	SERIAL0 (TXD)
4	PE2 (XCK0/AIN0)		
5	PE3 (OC3A/AIN1)	Digital pin 5 (PWM)	04
6	PE4(OC3B/INT4)	Digital pin 2 (PWM)	D2
7	PE5 (OC3C/INT5)	Digital pin 3 (PWM)	05
8	PE6 (T3/INT6)		
9	PE7 (CLK0/ICP3/INT7)		
10	VCC	VCC	
11	GND	GND	
12	PH0 (RXD2)	Digital pin 17 (RX2)	SERIAL2 (TXD)
13	PH1 (TXD2)	Digital pin 16 (TX2)	SERIAL2 (TXD)
14	PH2 (XCK2)		
14	PH3 (OC4A)	Digital pin 6 (PWM)	03
16	PH4 (OC4B)	Digital pin 7 (PWM)	D7
17	PH5 (OC4C)	Digital pin 8 (PWM)	D8
18	PH6 (OC2B)	Digital pin 9 (PWM)	02
19	PB0 (SS/PCINT0)	Digital pin 53 (SS)	
20	PB1 (SCK/PCINT1)	Digital pin 52 (SCK)	
21	PB2 (MOSI/PCINT2)	Digital pin 51 (MOSI)	
22	PB3 (MISO/PCINT3)	Digital pin 50 (MISO)	
23	PB4 (OC2A/PCINT4)	Digital pin 10 (PWM)	01

Pin number	Pin Name	Map Pin Name	Pin on TinkerKit
24	PB5 (OC1A/PCINT5)	Digital pin 11 (PWM)	O0
25	PB6 5OC1B/PCINT6)	Digital pin 12 (PWM)	D12
26	PB7 (OC0A/OC1C/PCINT7)	Digital pin 13 (PWM)	D13
27	PH7 (T4)		
28	PG3 (TOSC2)		
29	PG4 (TOSC1)		
30	RESET	RESET	
31	VCC	VCC	
32	GND	GND	
33	XTAL2	XTAL2	
34	XTAL1	XTAL1	
35	PL0 (ICP4)	Digital pin 49	
36	PL1 (ICP5)	Digital pin 48	
37	PL2 (T5)	Digital pin 47	
38	PL3 (OC5A)	Digital pin 46 (PWM)	
39	PL4 (OC5B)	Digital pin 45 (PWM)	
40	PL5 (OC5C)	Digital pin 44 (PWM)	
41	PL6	Digital pin 43	
42	PL7	Digital pin 42	
43	PD0 (SCL/INT0)	Digital pin 21 (SCL)	TWI (SCL)
44	PD1 (SDA/INT1)	Digital pin 20 (SDA)	TWI (SDA)
45	PD2 (RXD1/INT2)	Digital pin19 (RX1)	SERIAL1 (RXD)
46	PD3 (TXD1/INT3)	Digital pin 18 (TX1)	SERIAL1 (TXD)

Pin number	Pin Name	Map Pin Name	Pin on Tinker Kit
47	PD4 (ICP1)		
48	PD5 (XCK1)		
49	PD6 (T1)		
50	PD7 (T0)	Digital pin 38	
51	PG0 (WR)	Digital pin 41	
52	PG1 (RD)	Digital pin 40	
53	PC0 (A8)	Digital pin 37	
54	PC1 (A9)	Digital pin 36	
55	PC2 (A10)	Digital pin 35	
56	PC3 (A11)	Digital pin 34	
57	PC4 (A12)	Digital pin 33	
58	PC5 (A13)	Digital pin 32	
59	PC6 (A14)	Digital pin 31	
60	PC7 (A15)	Digital pin 30	
61	VCC	VCC	
62	GND	GND	
63	PJ0 (RXD3/PCINT9)	Digital pin 15 (RX3)	SERIAL3 (RXD)
64	PJ1 (TXD3/PCINT10)	Digital pin 14 (TX3)	SERIAL3 (TXD)
65	PJ2 (XCK3/PCINT11)		
66	PJ3 (PCINT12)		
67	PJ4 (PCINT13)		
68	PJ5 (PCINT14)		
69	PJ6 (PCINT15)		

Pin number	Pin Name	Map Pin Name	Pin on Tinker Kit
70	PG2 (ALE)	Digital pin 39	
71	PA7 (AD7)	Digital pin 29	
72	PA6 (AD6)	Digital pin 28	
73	PA5 (AD5)	Digital pin 27	
74	PA4 (AD4)	Digital pin 26	
75	PA3 (AD3)	Digital pin 25	
76	PA2 (AD2)	Digital pin 24	
77	PA1 (AD1)	Digital pin 23	
78	PA0 (AD0)	Digital pin 22	
79	PJ7		
80	VCC	VCC	
81	GND	GND	
82	PK7 (ADC15/PCINT23)	Analog pin 15	
83	PK6 (ADC14/PCINT22)	Analog pin 14	
84	PK5 (ADC13/PCINT21)	Analog pin 13	
85	PK4 (ADC12/PCINT20)	Analog pin 12	
86	PK3 (ADC11/PCINT19)	Analog pin 11	
87	PK2 (ADC10/PCINT18)	Analog pin 10	
88	PK1 (ADC9/PCINT17)	Analog pin 9	I9
89	PK0 (ADC8/PCINT16)	Analog pin 8	I8
90	PF7 (ADC7)	Analog pin 7	I7
91	PF6 (ADC6)	Anaog pin 6	I6
92	PF5 (ADC5/TMS)	Analog pin 5	I5

Pin number	Pin Name	Map Pin Name	Pin on Tinker Kit
93	PF4 (ADC4/TMK)	Analog pin 4	I4
94	PF3 (ADC3)	Analog pin 3	I3
95	PF2 (ADC2)	Analog pin 2	I2
96	PF1 (ADC1)	Analog pin 1	I1
97	PF0 (ADC0)	Analog pin 0	IO
98	AREF	Analog Reference	
99	GND	GND	
100	AVCC	VCC	